

Build Roadmap — Vibe Coding Pakka to Production

34df0157266f80c8aaaff0689d730ccf

Build Roadmap — Vibe Coding Pakka to Production



Goal: Ship a production-grade *open-source reference implementation* of milestone-based escrow for Indian marketplaces. Not a startup — a portfolio artifact that proves you can architect and ship the kind of work senior teams hire for: atomic state machines, RLS isolation, realtime sync, PWA delivery, Play Store packaging. Built almost entirely with AI coding tools.



Why this framing wins: The “escrow for blue-collar workers” *business* is hard for well-known reasons — cash-on-completion culture, RBI/PPI licensing, worker liquidity, crowded market. By shipping Pakka as **infrastructure code any marketplace could fork**, you sidestep every business risk while keeping 100% of the engineering showcase. Recruiters reviewing your GitHub see exactly the kind of work senior teams hire for. You ship in 4-6 weeks instead of fighting a market for years.

TL;DR — My recommendation

Question	Answer	Why
Web app or mobile app?	Both — build a PWA first, then wrap for Play Store	One codebase, web demo link for resume, Play Store install via TWA. Fastest path to “industry ready.”
Primary AI coding tool?	Dual-IDE setup: Antigravity (hard phases) + Trae (fast phases). Models: Claude Opus 4.6 for escrow + security audit; Sonnet 4.6 as everyday all-rounder; GPT-5.3-Codex on Trae for fast feature work; Gemini 3.1 Pro for huge-context refactors. Claude Code (terminal) for autonomous background runs.	Opus 4.6 leads SWE-bench Verified at 80.8% with 1M context — only model that holds the full escrow ledger + RLS surface coherently. Antigravity’s multi-agent workflow beats single-agent IDEs for the complex phases; Trae stays as the fast-iteration daily driver. This dual setup is what top solo devs are running in April 2026.
Backend?	Supabase (Postgres + Auth + Realtime + Storage + Edge Functions)	Industry-standard. Free tier covers your demo. Built-in row-level security = real auth.
Frontend stack?	Next.js 15 (App Router) + TypeScript + Tailwind + shadcn/ui	Hireable stack. PWA-ready. Vercel deploy in one click.
Mobile wrap?	PWABuilder → Trusted Web Activity (TWA) for Play Store	No need to rewrite in React Native unless you want to.
Payments / escrow?	Mock wallet by design , prominent “Demo Mode” banner	Real INR escrow needs an RBI/PPI license. The <i>engineering</i> (state machine, ledger atomicity, RLS) is 100% real. Document Razorpay Route swap as a 1-page production note for forkers.



The honest framing (this is a strength, not a weakness): Pakka is an **open-source reference implementation** of milestone escrow infrastructure. The state machine, ledger atomicity, RLS policies, dispute machinery, and realtime sync are 100% production-grade — the kind of code a real marketplace would fork. The money is

simulated *by design*, because real INR escrow requires an RBI-approved PPI license. This framing is *more* impressive to senior engineers than a half-real product, because it shows you understand both the regulatory ceiling AND the right engineering for inside it.

How this framing solves every business risk

Original concern	How Path A neutralizes it
Indian customers prefer cash on completion — escrow has no consumer pull	Not selling to consumers. Selling <i>the engineering</i> to recruiters and marketplace builders.
Workers won't onboard for KYC + 72h auto-release	Seeded demo workers cover every state. Zero acquisition needed for a reference implementation.
RBI / PPI license required for real money in India	Demo Mode is the <i>intended design</i> , not a workaround. Razorpay Route swap documented as a 1-page integration note for forkers.
Crowded market — WorkUnion, Bandhoo, Apna, Urban Company	Different category entirely. You're shipping infrastructure, not competing for users.
Solo dev can't get two-sided marketplace liquidity	Liquidity is irrelevant. The artifact is the GitHub repo + demo + writeups.
Escrow overkill for ₹2k–15k home services tickets	Seed the demo with ₹50k+ renovation tickets where milestone escrow is realistic.

- —

Why PWA + TWA beats native for your use case

- **One codebase** → ship faster, fewer bugs, easier to maintain solo.
- **Web demo link** → recruiters can click and try it without installing anything.
- **Play Store presence** → install button, push notifications, splash screen — looks identical to native.
- **Push notifications** work on both Android Chrome and PWA-installed apps via Web Push.
- **Photo/video uploads, geolocation, camera** all work in modern PWAs.
- **Skips the “learn React Native” detour** unless you specifically want native skills on resume.

If you later want to add it to your resume as “React Native expertise,” you can port the screens — the backend stays identical.

- —

The recommended stack (industry-ready, hireable in 2026)

Frontend

- **Next.js 15** (App Router, Server Components, Server Actions)
- **TypeScript** (non-negotiable for serious projects)
- **Tailwind CSS v4** + **shadcn/ui** for components
- **Zustand** for client state, **TanStack Query** for server state
- **next-pwa** for offline + install prompt
- **Web Push API** for notifications

Backend

- **Supabase**
 - **Postgres** for all data
 - **Auth** with phone OTP (Supabase has built-in Twilio/MSG91 integration)
 - **Row-Level Security (RLS)** policies — this is the real auth boundary
 - **Realtime** subscriptions for live escrow state changes
 - **Storage** for photo/video proof
 - **Edge Functions** (Deno) for state machine transitions and webhooks

Infra & DevOps

- **Vercel** for frontend (free, auto-deploys from GitHub)
- **Supabase Cloud** for backend (free tier)
- **GitHub Actions** for CI (lint, type-check, test on PR)
- **Sentry** for error tracking (free tier)
- **PostHog** for analytics + session replay (free tier)

Mobile packaging

- **PWABuilder.com** → generates signed Android bundle
- **Google Play Console** (\$25 one-time fee) for Play Store submission

Optional but resume-boosting

- **Stripe** or **Razorpay** test mode for payment UI realism
- **MSG91** for real OTP delivery in India
- **Cloudflare R2** if you outgrow Supabase storage
- —

Roadmap — 8 phases over ~4-6 weeks (part-time)

Phase 0 — Foundations (Day 1)

Goals: Public repo, accounts, design system, open-source positioning locked.

- ✓ Create ~~public~~ GitHub repo ~~pakka~~ with ~~MIT license~~, pin on profile
- ✓ Add ~~README.md~~ skeleton: 1-line tagline ("Production-grade open-source escrow infrastructure for Indian marketplaces"), placeholder badges, demo link placeholder
- ✓ Sign up: Vercel, Supabase, Sentry, PostHog, PWABuilder
- ✓ Buy a cheap domain (e.g. ~~mypakka.vercel.app~~) — recruiters love a custom domain
- ✓ Decide brand: deep blue ~~#0A2540~~, emerald ~~#10B981~~, neutral grays
- ✓ Pin ~~dual IDE setup~~: Antigravity (Sonnet 4.6 / Opus 4.6) for hard phases (4, 7) + Trae (GPT-5.3-Codex / Sonnet 4.6) for fast phases (3, 5, 6, 8). Claude Code in terminal as autonomous background agent. Full tool

stack is documented in the *Jaw Dropping AI Tool Stack* section below.

Phase 1 — Schema & state machine design (Day 2-3)

Goals: Get the data model and escrow state machine right BEFORE writing any UI. This is where most AI-built projects fail.

- ✓ Design Postgres schema (11 tables: profiles, worker_profiles, wallets, jobs, milestones, escrow_ledger, proofs, disputes, materials, job_applications, notifications)
- ✓ Define escrow state machine: `pending → funded → submitted → approved → released` with dispute branches and admin overrides
- ✓ Write RLS policies for every table (admin bypass via `is_admin()`, owner/participant scoped reads, SECURITY DEFINER for money writes)
- ✓ Seed migration with sample data (3 clients, 5 workers across KYC states, 4 jobs covering every milestone state)
- ✓ Hardening migration: column-locking triggers + `submit_milestone()` + admin KYC functions

Phase 2 — Auth + role bifurcation (Day 4-5)

- ✓ Phone OTP login screen
- ✓ Post-OTP role picker (Client / Worker) — only on first login
- ✓ Client onboarding: name, city
- ✓ Worker onboarding: KYC flow (Aadhaar number, PAN, selfie, work category, skill tags)
- ✓ Pending KYC state — workers can browse but not accept jobs

Phase 3 — Job posting + browse (Day 6-8)

- ✓ Client: post job with title, description, location, budget, milestones, materials list
- ✓ Worker: browse feed filtered by category + distance
- ✓ Worker: apply to job with bid + ETA
- ✓ Client: accept worker — triggers escrow funding screen (placeholder route)



Phase 3 lessons learned — gotchas the IDE buried (carry these into ADRs at Phase 8):

- **Demo-mode auth must use Supabase test phone numbers, not `signInAnonymously()`**. The first auth fix called `supabase.auth.signInAnonymously()` inside `verify0tpAction` when `DEMO_MODE=true`. That created a brand-new anon user on every login instead of authenticating as the actual phone number. RLS then evaluated against the wrong UID and the worker feed showed zero jobs even though the data was reachable. Fix: delete the demo short-circuit entirely; with test phone numbers configured in Supabase Auth → Phone, the regular `signInWith0tp` + `verify0tp` calls work without sending real SMS. → **ADR-0022 (queued)**.
- **`File.arrayBuffer()` must be awaited and the resulting `ArrayBuffer` passed to `supabase.storage.upload()` from a Server Action.** Passing the raw `File` object across the Server-Action boundary throws `TypeError: Failed to fetch`. → **ADR-0020 (queued)**.
- **KYC status reads must run server-side via `createServerClient` and be passed down as props, not fetched from a client component.** The original client-side `worker_profiles` query silently failed RLS (returned no row, defaulted to `pending`) because the browser session cookies weren't being threaded correctly. Server component now reads `kyc_status` and passes it as `workerKyc` to `<WorkerFeed/>`. → **ADR-0021 (queued)**.

- **hasApplied check must be scoped per-job, not global.** Caught on prod `998ad36`: Ravi applied to *Kitchen sink leak fix*, then opened *Drainage repair* and saw "Application sent". The worker job-detail existence check filtered `job_applications` by `worker_id` only, so any single application poisoned every other job's CTA. Fix: add `.eq('job_id', jobId)` to the detail-page check; for the feed list, fetch the worker's applied `job_id`s as a `Set<string>` once and check `appliedJobIds.has(job.id)` per card — never a single boolean. Shipped as `577400d`. → **ADR-0024 (queued)**.
- **Validate fixes against clean test data, not stale rows from earlier buggy sessions.** After shipping `577400d` (the per-job scoping fix), the same bug appeared to reproduce. Real cause: the buggy code on `998ad36` had let Ravi insert a real `job_applications` row against Drainage repair on Apr 28. The fix was correct — the data was just polluted. Always run a `SELECT \* FROM <table> WHERE <test_user>` before declaring a fix broken. Add a one-line dev script or saved Supabase SQL query for resetting test-user data between sessions. → **ADR-0026 (queued)**.
- **Demo-mode auth must use Supabase test phone numbers, not `signInAnonymously()`.** The first auth fix called `supabase.auth.signInAnonymously()` inside `verifyOtpAction` when `DEMO_MODE=true`. That created a brand-new anon user on every login instead of authenticating as the actual phone number. RLS then evaluated against the wrong UID and the worker feed showed zero jobs even though the data was reachable. Fix: delete the demo short-circuit entirely; with test phone numbers configured in Supabase Auth → Phone, the regular `signInWithOtp` + `verifyOtp` calls work without sending real SMS. → **ADR-0022 (queued)**.
- **`File.arrayBuffer()` must be awaited and the resulting `ArrayBuffer` passed to `supabase.storage.upload()` from a Server Action.** Passing the raw `File` object across the Server-Action boundary throws `TypeError: Failed to fetch`. → **ADR-0020 (queued)**.
- **KYC status reads must run server-side via `createServerClient` and be passed down as props, not fetched from a client component.** The original client-side `worker_profiles` query silently failed RLS (returned no row, defaulted to `pending`) because the browser session cookies weren't being threaded correctly. Server component now reads `kyc_status` and passes it as `workerKyc` to `<WorkerFeed/>`. → **ADR-0021 (queued)**.
- **hasApplied check must be scoped per-job, not global.** Caught on prod `998ad36`: Ravi applied to *Kitchen sink leak fix*, then opened *Drainage repair* and saw "Application sent". The worker job-detail existence check filtered `job_applications` by `worker_id` only, so any single application poisoned every other job's CTA. Fix: add `.eq('job_id', jobId)` to the detail-page check; for the feed list, fetch the worker's applied `job_id`s as a `Set<string>` once and check `appliedJobIds.has(job.id)` per card — never a single boolean. Shipped as `577400d`. → **ADR-0024 (queued)**.
- **Validate fixes against clean test data, not stale rows from earlier buggy sessions.** After shipping `577400d` (the per-job scoping fix), the same bug appeared to reproduce. Real cause: the buggy code on `998ad36` had let Ravi insert a real `job_applications` row against Drainage repair on Apr 28. The fix was correct — the data was just polluted. Always run a `SELECT \* FROM <table> WHERE <test_user>` before declaring a fix broken. Add a one-line dev script or saved Supabase SQL query for resetting test-user data between sessions. → **ADR-0026 (queued)**.



Phase 7 followups (do not lose these):

- **`src/middleware.ts` is missing.** Phase 2's prompt required it for Supabase session-cookie refresh between requests. The earlier IDE report falsely claimed it was audited. Recreate during the Phase 7 security pass with the standard `@supabase/ssr` middleware that calls `supabase.auth.getUser()` to refresh tokens.
- **`milestones` table SELECT RLS gap.** Worker job detail shows `Milestones (0)` because the policy doesn't grant access to non-assigned workers viewing an open job. Patch with a participant-aware policy: workers on the job (or any authenticated user for `status='open'` parent jobs, depending on product call) should see milestone

rows. Bake into the consolidated `00XX_fix_onboarding_rls.sql` migration in Phase 7.

- **✓ Client job detail Applications (0) — FIXED Apr 30 (commit on prod).** Root cause: `src/app/client/jobs/[id]/client-job-detail.tsx` ran a `Promise.all` whose `job_applications` SELECT embedded two joins via the same FK hint — `profiles!job_applications\worker_id\_fkey(full\_name)` (valid) AND `worker_profiles!job_applications\worker_id\_fkey(trust\_tier)` (invalid — no FK from `job_applications` → `worker_profiles` by that name; that table is reached via `worker_profiles\profile_id\_fkey → profiles.id`). PostgreSQL 400'd the whole request. The fetcher only checked `jobRes.error`, so `appRes.error` was swallowed and `(appRes.data ?? []).map(...)` returned `[]`. RLS was a red herring; the count badge worked because it uses a different code path with no broken embed. **Fix shipped:** dropped both embedded joins, fetch `job_applications` alone, enrich client-side with two `.in('id', workerIds)` lookups against `profiles` and `worker_profiles`; added `if (appRes.error) throw appRes.error;` (and equivalents for `msRes` / `matRes`) so future PostgreSQL embed bugs fail loudly instead of rendering zero. Verified on prod: Drainage repair detail now shows both applications (Ravi pending ₹15,000 + other rejected ₹12,000). → **ADR-0032: every branch of Promise.all must check its own error — silent ?? [] defaults turn 400s into invisible empty UI.** **ADR-0033 (new): when an embedded join would cross more than one parent table, prefer two sequential .in(ids) lookups over speculative FK-hinted embeds; FK hints only resolve to the table the constraint actually references.**

- **Wallet LOCKED card vs breakdown mismatch — ₹20,000 drift.** Discovered Apr 30 smoke Step 2. After funding Drainage milestone 1 (₹15k) on Aarav's wallet, `/client/wallet` shows AVAILABLE ₹1,35,000 / LOCKED ₹55,000, but the Locked Breakdown rows sum to ₹75,000 (2BHK kitchen renovation ₹60,000 + Drainage repair ₹15,000). `wallets.locked\_balance` aggregate column is drifting from the per-job rollup that `milestones WHERE status='funded'` produces. Likely the seed initialised `wallets.locked\_balance` at ₹40k for the kitchen renovation while the `milestones` table actually has ₹60k worth funded — Phase 4 RPCs only top-up the column for new funds, never reconcile. SQL audit before Phase 5: `SELECT locked\_balance FROM wallets WHERE profile\_id='aarav'` vs `SELECT job\_id, SUM(amount) FROM milestones WHERE status='funded' AND job\_id IN (SELECT id FROM jobs WHERE client\_id='aarav')` GROUP BY `job\_id`. Fix path: either (a) make Locked Breakdown read directly from `wallets` and a per-job derived view, OR (b) drop the `wallets.locked\_balance` column and compute it from `milestones` on read — single source of truth either way. → **ADR-0035 (queued): wallet aggregate columns and per-job rollups must be reconciled by SQL invariant tests; add a fast-check property asserting SUM(milestones.amount WHERE status='funded') = wallets.locked_balance for every client wallet, run in the Phase 7 test suite.**

- **Worker name renders as placeholder "Worker" instead of profiles.full_name.** On `/client/jobs/[id]` application cards (post- `Applications (0)` fix on prod) both Ravi and the rejected applicant render with the literal string "Worker" as the display name. Root cause: the new `client-job-detail.tsx` enriches applications with `.from('profiles').in('id', workerIds)` from the **browser** client session — `profiles` RLS only permits a user to read their own row, so other workers' rows return empty and the `?? "Worker"` fallback fires for every card. Fix: expose worker name+trust_tier via either (a) a `SECURITY DEFINER` RPC `get\_application\_worker\_summary(worker\_ids uuid[])` that returns `{ id, full\_name, trust\_tier }` only for workers the calling client has an active `job_applications` row with, OR (b) a scoped RLS read policy on `profiles.full\_name` for any user connected via a non-rejected `job_applications` row. → **ADR-0034 (queued): client UIs displaying counterparty names must go through a SECURITY DEFINER RPC or a scoped RLS policy — plain client-session .from('profiles').in('id', ...) will be RLS-empty.**

- **Trust tier badge UI vs DB mismatch (Ravi).** UI shows "Bronze" badge on Ravi's application card, but `worker_profiles.trust\_tier` for Ravi is `gold` in the DB. Same root cause as the worker-name issue: the client session's `.from('worker_profiles').in('profile\_id', workerIds)` lookup returns empty under RLS, so the badge component falls back to its default "bronze". Audit `src/components/ui/status-badge.tsx` (or wherever the trust-tier badge lives) to surface a `null` state explicitly instead of silently rendering bronze, and fix the data path via the same RPC/RLS-scoped-policy from ADR-0034.

- **✗ Realtime subscription is BROKEN — confirmed Apr 30 smoke Step 3.** Sequence: Ravi submitted

milestone 1 (Drainage repair, ₹15k) at ~02:25 IST. Aarav's `/client/jobs/094b2856-.../milestones` tab was already open with the page rendered showing `Locked In Escrow`. Without refreshing, the badge stayed `Locked In Escrow` indefinitely — no flip, no toast, no Approve button surfaced. After hard refresh, the page correctly rendered `Awaiting Review` (amber) with Worker submitted 7m ago / Auto-releases just now if no action taken / Approve + Dispute CTAs — confirming the DB state is correct (`milestones.status='submitted'` , `submitted_at` set, trigger fired into `notifications`). The bug is purely on the **client-side Realtime channel** not delivering the UPDATE event to the open tab. Likely root causes to investigate in priority order: (1) **Channel filter scope** — the milestone subscription may be filtering by `job_id=eq.{jobId}` but Supabase Realtime requires the filter column to be in the publication's REPLICA IDENTITY; if `milestones` isn't `REPLICA IDENTITY FULL` or the `job_id` column isn't in the default identity, filtered subscriptions silently drop events. Check `ALTER TABLE milestones REPLICA IDENTITY FULL` and the `supabase_realtime` publication config: `SELECT \* FROM pg_publication_tables WHERE pubname='supabase_realtime' AND tablename='milestones'`. (2) **Auth context on the subscription** — if the channel was opened with the anon key instead of the authenticated session token, RLS will block the `postgres_changes` payload from reaching the client (Supabase Realtime evaluates the SELECT policy against the subscriber's JWT). Verify the client passes the user's session in `supabase.realtime.setAuth(session.access_token)` after `signInWithOtp` resolves, and that the channel is created from the same client instance. (3) **Channel cleanup race** — if the milestones page mounts a channel and a hot-reload or React-strict-mode double-mount tears it down before the subscription actually finishes the WebSocket handshake, the second mount may attach to a dead channel. Add `channel.subscribe((status) => console.log(...))` and verify status reaches `SUBSCRIBED`. (4) **Filter syntax bug** — Supabase's filter is `filter: { job_id: jobId }`; an object form silently no-ops. Files to grep: `src/app/client/jobs/[id]/milestones/page.tsx`, `src/app/worker/jobs/[id]/milestones/page.tsx`, and any `useEffect` calling `supabase.channel(...)`. → **ADR-0037 (queued): Realtime subscriptions must (a) call `setAuth` with the authenticated session, (b) verify the table is in `supabase_realtime` publication with `REPLICA IDENTITY FULL`, (c) log subscribe status until `SUBSCRIBED`, (d) include a manual-refresh fallback button for users when the WebSocket drops.** Until fixed, every Phase 4 happy-path UI must surface a polling fallback or a "Refresh" hint when stale data is suspected. **Smoke is NOT blocked** — proceed with Approve via the refreshed page; Realtime fix gets its own commit + verification pass before Phase 5.

- **Worker has no nav link to `/worker/applications` — assigned jobs are invisible. ADR-0036 NOW AT 4 STRIKES** (Apr 30, 2026 ≈04:54 PM IST). Strike #4: hard-refreshed `/worker/jobs/2b86658c-.../milestones`, clicked the in-app *Browse* / back affordance → routed to `/worker/jobs` → **404 This page could not be found**. The route literally does not exist (no `src/app/worker/jobs/page.tsx`); workers' "my jobs" surface is `/worker/applications`. Strikes recap: (1) drainage smoke — Ravi typed `/worker/applications` manually; (2) smoke #2 post-OTP login dumped Ravi on `/worker/feed` with assigned smoke job invisible; (3) smoke #2 mid-flow same; (4) **NOW** — back-button orphan to `/worker/jobs` 404. Fix shipping in the combined next-commit alongside H2/H3/H4 Realtime + ADR-0034 worker-name RPC: (a) create `src/app/worker/jobs/page.tsx` that `redirect("/worker/applications")`, (b) audit all back/breadcrumb buttons under `src/app/worker/jobs/[id]/\*` to point to `/worker/applications` not `/worker/jobs`, (c) ship `<WorkerNavShell>` + `<ClientNavShell>` as sticky bottom-bar layouts under `src/app/worker/layout.tsx` and `src/app/client/layout.tsx` so every authenticated route group has Browse / My Jobs / Wallet always visible. Discovered Apr 30 smoke Step 3 (Ravi POV). After Aarav accepted Ravi for Drainage repair, Ravi logged in and `/worker/feed` showed only the two remaining `status='open'` jobs (Kitchen sink leak fix, Carpentry shelving) — zero indication that Drainage repair was assigned to him. The `/worker/applications` route exists and works perfectly (Accepted (1) section renders Drainage repair with bid ₹15k / 5d ETA), but **nothing in the worker UI links to it**. Had to type the URL manually. A real worker would think the app dropped their accepted job. Two fixes needed: (1) add a top-level worker nav (tabs or sidebar) with at minimum **Browse • My Applications • My Wallet**, present on every `/worker/\*` route; (2) on `/worker/feed`, surface a sticky banner or top section "You have N active job(s)" linking to `/worker/applications` whenever the worker has any non-rejected applications. The feed itself should keep showing only open jobs (correct product call), but it must not be the worker's only landing page. → **ADR-0036 (queued): every authenticated route group (`/client/\*`, `/worker/\*`, `/admin/\*`)**

must ship with a persistent navigation shell exposing every primary surface in that role; “the route exists” is not the same as “the user can reach it.” Audit the client side too — confirm `/client/wallet`, `/client/jobs`, `/client/jobs/new` are all reachable from a single nav, not just from each other.

- **Worker job-detail** (`/worker/jobs/[id]`) shows **Milestones (1)** for Ravi (assigned worker) but **Milestones (0)** for non-assigned workers viewing an open job. This is now confirmed two-sided: the assigned-worker case works (Ravi sees the milestone with ₹15k + Locked In Escrow badge → Submit for Review CTA), but the existing Phase 7 followup about non-assigned workers seeing **Milestones (0)** still stands. The fix is the same RLS patch already queued, just confirming the gap is **only** for non-participants — assigned workers can read their milestones correctly.

- **Dashboard greeting hardcoded as “Dashboard”** — must personalize with the user’s name. Reported Apr 30, 2026 ≈03:25 IST. Both `/client` and `/worker` landing pages (and presumably `/admin`) render the page header as the literal string “Dashboard” instead of greeting the logged-in user — e.g., should be “Welcome back, Aarav” on `/client` or “Hi Ravi 🙌” on `/worker`. Tiny fix, high product-feel signal: personalization is what separates a marketplace from a CRUD demo, and it’s the first thing a recruiter sees after login on the demo video. Files to edit: `src/app/client/page.tsx`, `src/app/worker/page.tsx`, `src/app/admin/page.tsx` (or wherever each role’s dashboard heading lives). Implementation: pull `profiles.full_name` server-side via the existing `createServerClient` session (zero extra round-trip — the layout already loads the profile for the nav), pass as prop to the dashboard component, render as `<h1>Welcome back, {firstName}</h1>` with `firstName = full_name?.split(' ')[0] ?? null`. Empty-name fallback: plain “Welcome back” (never show the bare “Dashboard” again). Bonus polish for the demo video: time-of-day variant — *Good morning / Good afternoon / Good evening* based on the user’s local hour (Asia/Kolkata for now, swap to `profiles.timezone` later). Add a small subtitle under the greeting: client → “You have *N* active job(s)”; worker → “You have *N* active application(s)”; admin → “*N* open disputes · *N* pending KYCs”. The subtitle doubles as the navigation hint that solved the orphan-URL problem (ADR-0036). → **ADR-0038 (queued): authenticated landing pages must personalize the heading from `profiles.full_name` and surface a one-line role-specific status subtitle. “Dashboard” is a placeholder string, not a header — it should never ship to prod. Audit every authenticated route group for this pattern before Phase 8. Update Apr 30, 2026 ≈03:38 IST: full spec for both the login moment AND the personalized greeting now lives in Phase 6.5 of this roadmap (see above). ADR-0039 supersedes ADR-0038 with richer scope: live escrow ticker on `/login`, two-door (hire / earn) entry, time-banded greeting copy library (CLIENT × 4 bands, WORKER × 4 bands, ADMIN), deterministic daily quote rotator with 8 Pakka-specific lines, en+hi i18n from day one, snapshot-tested. The bare “Dashboard” string and the generic “Enter your phone number” form both die in the same commit.**

- **✓ Form CTA escape bug — FIXED May 1, 2026 ≈01:08 IST** on PR #1 branch `fix/realtime-nav-shell-worker-summary` (commit on top of `e70a9d9`). Reported on `/client/jobs/new` step 1: Next button rendered behind the sticky `<ClientNavShell>` bottom nav. DevTools bounding rect pre-fix: `top=537.33 / bottom=569.33 / viewportHeight=585` — only 16px between button bottom and viewport edge, fully occluded by the 56px `h-14` nav. Root cause: `post-job-form.tsx`’s sticky CTA wrapper used `fixed inset-x-0 bottom-0 z-20`, sitting on top of the 56px nav (which itself uses `fixed bottom-0 z-50`); with `z-50 > z-20`, the nav stacked over the CTA and ate the bottom 56px. Two-line fix: `bottom-0` → `bottom-14` on the CTA wrapper (offsets the CTA above the nav), and form padding `pb-28` → `pb-36` for clearance against both bars. Verified live on Vercel preview `pakka-3ce1wtv2q-shaiksohellls-projects.vercel.app/client/jobs/new` — button visible, clickable, no overlap. → **ADR-0043 (queued): when a route group ships a sticky bottom nav shell (ADR-0036), every fixed-bottom CTA inside that route group must be offset by the nav’s height. `fixed inset-x-0 bottom-0` under a nav-shell layout is forbidden — use `bottom-{nav-height}` (e.g., `bottom-14` for an `h-14` nav) and pad the scrollable container by both the CTA height AND the nav height. Audit checklist: `grep fixed inset-x-0 bottom-0` across `src/app/client/*` and `src/app/worker/*` after any nav-shell change. Lesson logged: `position: fixed` ignores parent padding — when a UI is reported as “hidden behind” something, check z-index stacking and bottom-offset BEFORE blaming layout/padding. GitHub**

Copilot's commit-title suggestions are vague ("Adjust padding in post job form layout") and must always be overridden with a proper conventional-commit message.

- **src/middleware.ts is missing.** Phase 2's prompt required it for Supabase session-cookie refresh between requests. The earlier IDE report falsely claimed it was audited. Recreate during the Phase 7 security pass with the standard `@supabase/ssr` middleware that calls `supabase.auth.getUser()` to refresh tokens.
- **milestones table SELECT RLS gap.** Worker job detail shows `Milestones (0)` because the policy doesn't grant access to non-assigned workers viewing an open job. Patch with a participant-aware policy: workers on the job (or any authenticated user for `status='open'` parent jobs, depending on product call) should see milestone rows. Bake into the consolidated `00XX_fix_onboarding_rls.sql` migration in Phase 7.
- **✓ Client job detail Applications (0) — FIXED Apr 30 (commit on prod).** Root cause: `src/app/client/jobs/[id]/client-job-detail.tsx` ran a `Promise.all` whose `job_applications` SELECT embedded two joins via the same FK hint — `profiles!job_applications\worker_id_fkey(full_name)` (valid) AND `worker_profiles!job_applications\worker_id_fkey(trust_tier)` (invalid — no FK from `job_applications` → `worker_profiles` by that name; that table is reached via `worker_profiles\profile_id_fkey → profiles.id`). PostgreSQL 400'd the whole request. The fetcher only checked `jobRes.error`, so `appRes.error` was swallowed and `(appRes.data ?? []).map(...)` returned `[]`. RLS was a red herring; the count badge worked because it uses a different code path with no broken embed. **Fix shipped:** dropped both embedded joins, fetch `job_applications` alone, enrich client-side with two `.in('id', workerIds)` lookups against `profiles` and `worker_profiles`; added `if (appRes.error) throw appRes.error;` (and equivalents for `msRes / matRes`) so future PostgreSQL embed bugs fail loudly instead of rendering zero. Verified on prod: Drainage repair detail now shows both applications (Ravi pending ₹15,000 + other rejected ₹12,000). → **ADR-0032: every branch of Promise.all must check its own error — silent ?? [] defaults turn 400s into invisible empty UI. ADR-0033 (new): when an embedded join would cross more than one parent table, prefer two sequential .in(ids) lookups over speculative FK-hinted embeds; FK hints only resolve to the table the constraint actually references.**
- **Wallet LOCKED card vs breakdown mismatch — ₹20,000 drift.** Discovered Apr 30 smoke Step 2. After funding Drainage milestone 1 (₹15k) on Aarav's wallet, `/client/wallet` shows AVAILABLE ₹1,35,000 / LOCKED ₹55,000, but the Locked Breakdown rows sum to ₹75,000 (2BHK kitchen renovation ₹60,000 + Drainage repair ₹15,000). `wallets.locked_balance` aggregate column is drifting from the per-job rollup that `milestones WHERE status='funded'` produces. Likely the seed initialised `wallets.locked_balance` at ₹40k for the kitchen renovation while the `milestones` table actually has ₹60k worth funded — Phase 4 RPCs only top-up the column for new funds, never reconcile. SQL audit before Phase 5: `SELECT locked_balance FROM wallets WHERE profile_id='<aarav>'` VS `SELECT job_id, SUM(amount) FROM milestones WHERE status='funded' AND job_id IN (SELECT id FROM jobs WHERE client_id='<aarav>') GROUP BY job_id`. Fix path: either (a) make Locked Breakdown read directly from `wallets` and a per-job derived view, OR (b) drop the `wallets.locked_balance` column and compute it from `milestones` on read — single source of truth either way. → **ADR-0035 (queued): wallet aggregate columns and per-job rollups must be reconciled by SQL invariant tests; add a fast-check property asserting `SUM(milestones.amount WHERE status='funded') = wallets.locked_balance` for every client wallet, run in the Phase 7 test suite.**
- **Worker name renders as placeholder "Worker" instead of profiles.full_name.** On `/client/jobs/[id]` application cards (post-`Applications (0)` fix on prod) both Ravi and the rejected applicant render with the literal string "Worker" as the display name. Root cause: the new `client-job-detail.tsx` enriches applications with `.from('profiles').in('id', workerIds)` from the **browser** client session — `profiles` RLS only permits a user to read their own row, so other workers' rows return empty and the `?? "Worker"` fallback fires for every card. Fix: expose worker name+trust_tier via either (a) a `SECURITY DEFINER` RPC `get_application_worker_summary(worker_ids uuid[])` that returns `{ id, full_name, trust_tier }` only for workers

the calling client has an active `job\_applications` row with, OR (b) a scoped RLS read policy on `profiles.full\_name` for any user connected via a non-rejected `job\_applications` row. → **ADR-0034 (queued): client UIs displaying counterparty names must go through a SECURITY DEFINER RPC or a scoped RLS policy — plain client-session `.from('profiles').in('id', ...)` will be RLS-empty.**

- **Trust tier badge UI vs DB mismatch (Ravi).** UI shows “Bronze” badge on Ravi’s application card, but `worker\_profiles.trust\_tier` for Ravi is `gold` in the DB. Same root cause as the worker-name issue: the client session’s `.from('worker\_profiles').in('profile\_id', workerIds)` lookup returns empty under RLS, so the badge component falls back to its default “bronze”. Audit `src/components/ui/status-badge.tsx` (or wherever the trust-tier badge lives) to surface a `null` state explicitly instead of silently rendering bronze, and fix the data path via the same RPC/RLS-scoped-policy from ADR-0034.
- **✗ Realtime subscription is BROKEN — confirmed Apr 30 smoke Step 3.** Sequence: Ravi submitted milestone 1 (Drainage repair, ₹15k) at ~02:25 IST. Aarav’s `/client/jobs/094b2856-.../milestones` tab was already open with the page rendered showing `Locked In Escrow`. Without refreshing, the badge stayed `Locked In Escrow` indefinitely — no flip, no toast, no Approve button surfaced. After hard refresh, the page correctly rendered `Awaiting Review` (amber) with Worker submitted 7m ago / Auto-releases just now if no action taken / Approve + Dispute CTAs — confirming the DB state is correct (`milestones.status='submitted'`, `submitted\_at` set, trigger fired into `notifications`). The bug is purely on the **client-side Realtime channel** not delivering the UPDATE event to the open tab. Likely root causes to investigate in priority order: (1) **Channel filter scope** — the milestone subscription may be filtering by `job\_id=eq.{jobId}` but Supabase Realtime requires the filter column to be in the publication’s REPLICA IDENTITY; if `milestones` isn’t `REPLICA IDENTITY FULL` or the `job\_id` column isn’t in the default identity, filtered subscriptions silently drop events. Check `ALTER TABLE milestones REPLICA IDENTITY FULL` and the `supabase\_realtime` publication config: `SELECT \* FROM pg\_publication\_tables WHERE pubname='supabase\_realtime' AND tablename='milestones'`. (2) **Auth context on the subscription** — if the channel was opened with the anon key instead of the authenticated session token, RLS will block the `postgres\_changes` payload from reaching the client (Supabase Realtime evaluates the SELECT policy against the subscriber’s JWT). Verify the client passes the user’s session in `supabase.realtime.setAuth(session.access\_token)` after `signInWithOtp` resolves, and that the channel is created from the same client instance. (3) **Channel cleanup race** — if the milestones page mounts a channel and a hot-reload or React-strict-mode double-mount tears it down before the subscription actually finishes the WebSocket handshake, the second mount may attach to a dead channel. Add `channel.subscribe((status) => console.log(...))` and verify status reaches `SUBSCRIBED`. (4) **Filter syntax bug** — Supabase’s filter is `filter: 'job\_id=eq.<uid>'` not `filter: { job\_id: jobId }`; an object form silently no-ops. Files to grep: `src/app/client/jobs/[id]/milestones/page.tsx`, `src/app/worker/jobs/[id]/milestones/page.tsx`, and any `useEffect` calling `supabase.channel(...)`. → **ADR-0037 (queued): Realtime subscriptions must (a) call `setAuth` with the authenticated session, (b) verify the table is in `supabase\_realtime` publication with `REPLICA IDENTITY FULL`, (c) log subscribe status until `SUBSCRIBED`, (d) include a manual-refresh fallback button for users when the WebSocket drops.** Until fixed, every Phase 4 happy-path UI must surface a polling fallback or a “Refresh” hint when stale data is suspected. **Smoke is NOT blocked** — proceed with Approve via the refreshed page; Realtime fix gets its own commit + verification pass before Phase 5.
- **Worker has no nav link to `/worker/applications` — assigned jobs are invisible. ADR-0036 NOW AT 4 STRIKES** (Apr 30, 2026 ~04:54 PM IST). Strike #4: hard-refreshed `/worker/jobs/2b86658c-.../milestones`, clicked the in-app *Browse* / back affordance → routed to `/worker/jobs` → **404 This page could not be found**. The route literally does not exist (no `src/app/worker/jobs/page.tsx`); workers’ “my jobs” surface is `/worker/applications`. Strikes recap: (1) drainage smoke — Ravi typed `/worker/applications` manually; (2) smoke #2 post-OTP login dumped Ravi on `/worker/feed` with assigned smoke job invisible; (3) smoke #2 mid-flow same; (4) **NOW** — back-button orphan to `/worker/jobs` 404. Fix shipping in the combined next-commit alongside H2/H3/H4 Realtime + ADR-0034 worker-name RPC: (a) create `src/app/worker/jobs/page.tsx`

that `redirect("/worker/applications")`, (b) audit all back/breadcrumb buttons under `src/app/worker/jobs/[id]/\*` to point to `/worker/applications` not `/worker/jobs`, (c) ship `<WorkerNavShell>` + `<ClientNavShell>` as sticky bottom-bar layouts under `src/app/worker/layout.tsx` and `src/app/client/layout.tsx` so every authenticated route group has Browse / My Jobs / Wallet always visible. Discovered Apr 30 smoke Step 3 (Ravi POV). After Aarav accepted Ravi for Drainage repair, Ravi logged in and `/worker/feed` showed only the two remaining `status='open'` jobs (Kitchen sink leak fix, Carpentry shelving) — zero indication that Drainage repair was assigned to him. The `/worker/applications` route exists and works perfectly (Accepted (1) section renders Drainage repair with bid ₹15k / 5d ETA), but **nothing in the worker UI links to it**. Had to type the URL manually. A real worker would think the app dropped their accepted job. Two fixes needed: (1) add a top-level worker nav (tabs or sidebar) with at minimum **Browse • My Applications • My Wallet**, present on every `/worker/\*` route; (2) on `/worker/feed`, surface a sticky banner or top section “You have N active job(s)” linking to `/worker/applications` whenever the worker has any non-rejected applications. The feed itself should keep showing only open jobs (correct product call), but it must not be the worker's only landing page. → **ADR-0036 (queued): every authenticated route group (`/client/\*`, `/worker/\*`, `/admin/\*`) must ship with a persistent navigation shell exposing every primary surface in that role; “the route exists” is not the same as “the user can reach it.” Audit the client side too — confirm `/client/wallet`, `/client/jobs`, `/client/jobs/new` are all reachable from a single nav, not just from each other.**

- **Worker job-detail (`/worker/jobs/[id]`) shows Milestones (1) for Ravi (assigned worker) but Milestones (0) for non-assigned workers viewing an open job.** This is now confirmed two-sided: the assigned-worker case works (Ravi sees the milestone with ₹15k + Locked In Escrow badge → Submit for Review CTA), but the existing Phase 7 followup about non-assigned workers seeing `Milestones (0)` still stands. The fix is the same RLS patch already queued, just confirming the gap is **only** for non-participants — assigned workers can read their milestones correctly.
- **Dashboard greeting hardcoded as “Dashboard” — must personalize with the user’s name.** Reported Apr 30, 2026 ≈03:25 IST. Both `/client` and `/worker` landing pages (and presumably `/admin`) render the page header as the literal string “Dashboard” instead of greeting the logged-in user — e.g., should be “Welcome back, Aarav” on `/client` or “Hi Ravi 🙌” on `/worker`. Tiny fix, high product-feel signal: personalization is what separates a marketplace from a CRUD demo, and it's the first thing a recruiter sees after login on the demo video. Files to edit: `src/app/client/page.tsx`, `src/app/worker/page.tsx`, `src/app/admin/page.tsx` (or wherever each role's dashboard heading lives). Implementation: pull `profiles.full_name` server-side via the existing `createServerClient` session (zero extra round-trip — the layout already loads the profile for the nav), pass as prop to the dashboard component, render as `<h1>Welcome back, {firstName}</h1>` with `firstName = full_name?.split(' ')[0] ?? null`. Empty-name fallback: plain “Welcome back” (never show the bare “Dashboard” again). Bonus polish for the demo video: time-of-day variant — *Good morning / Good afternoon / Good evening* based on the user's local hour (Asia/Kolkata for now, swap to `profiles.timezone` later). Add a small subtitle under the greeting: client → “You have N active job(s)”; worker → “You have N active application(s)”; admin → “N open disputes · N pending KYCs”. The subtitle doubles as the navigation hint that solved the orphan-URL problem (ADR-0036). → **ADR-0038 (queued): authenticated landing pages must personalize the heading from `profiles.full_name` and surface a one-line role-specific status subtitle. “Dashboard” is a placeholder string, not a header — it should never ship to prod. Audit every authenticated route group for this pattern before Phase 8. Update Apr 30, 2026 ≈03:38 IST:** full spec for both the login moment AND the personalized greeting now lives in **Phase 6.5** of this roadmap (see above). **ADR-0039 supersedes ADR-0038** with richer scope: live escrow ticker on `/login`, two-door (hire / earn) entry, time-banded greeting copy library (CLIENT × 4 bands, WORKER × 4 bands, ADMIN), deterministic daily quote rotator with 8 Pakka-specific lines, en+hi i18n from day one, snapshot-tested. The bare “Dashboard” string and the generic “Enter your phone number” form both die in the same commit.

- ☒ **Form CTA escape bug — FIXED May 1, 2026 ≈01:08 IST** on PR #1 branch `fix/realtime-nav-shell-worker-summary` (commit on top of `e70a9d9`). Reported on `/client/jobs/new` step 1: Next button rendered behind the sticky `<ClientNavShell>` bottom nav. DevTools bounding rect pre-fix: `top=537.33 / bottom=569.33 / viewportHeight=585` — only 16px between button bottom and viewport edge, fully occluded by the 56px `h-14` nav. Root cause: `post-job-form.tsx`'s sticky CTA wrapper used `fixed inset-x-0 bottom-0 z-20`, sitting on top of the 56px nav (which itself uses `fixed bottom-0 z-50`); with `z-50 > z-20`, the nav stacked over the CTA and ate the bottom 56px. Two-line fix: `bottom-0` → `bottom-14` on the CTA wrapper (offsets the CTA above the nav), and form padding `pb-28` → `pb-36` for clearance against both bars. Verified live on Vercel preview `pakka-3ce1wtv2q-shaiksohell11s-projects.vercel.app/client/jobs/new` — button visible, clickable, no overlap. → **ADR-0043 (queued): when a route group ships a sticky bottom nav shell (ADR-0036), every fixed-bottom CTA inside that route group must be offset by the nav's height.** `fixed inset-x-0 bottom-0` under a nav-shell layout is forbidden — use `bottom-{nav-height}` (e.g., `bottom-14` for an `h-14` nav) and pad the scrollable container by both the CTA height AND the nav height. Audit checklist: `grep fixed inset-x-0 bottom-0` across `src/app/client/\*` and `src/app/worker/\*` after any nav-shell change. Lesson logged: `position: fixed` ignores parent padding — when a UI is reported as “hidden behind” something, check z-index stacking and bottom-offset BEFORE blaming layout/padding. GitHub Copilot's commit-title suggestions are vague (“Adjust padding in post job form layout”) and must always be overridden with a proper conventional-commit message.

Phase 4 — Escrow ledger + state machine (Day 9-12) ★ Hardest part

- ☐ Mock wallet top-up screen (with prominent “Demo Mode” banner)
- ☒ Fund escrow on job acceptance — atomic ledger entry
- ☐ Worker submits milestone proof (photo + video + notes)
- ☒ Client approve/dispute UI
- ☒ Approve → atomic ledger transfer → realtime UI update on both sides → push notification
- ☐ Worker withdrawable balance + mock withdrawal
- ☒ Auto-release timer (Edge Function cron)



Phase 4 ☒ shipped (Apr 30, 2026) — Antigravity + Opus 4.6 in one session. Commits on `main`: `193b1f3` (docs), `1681a94` (escrow core: schemas, Server Actions, milestone UIs, wallet pages), `3a32d60` (Vitest + fast-check tests), `c412957` (auto-release Edge Function + idempotency migration + notification trigger).

Headline wins:

- Full state machine end-to-end: fund → submit → approve → dispute → auto-release, every transition via existing **SECURITY DEFINER RPCs** (`fund_escrow`, `submit_milestone`, `approve_milestone`, `dispute_milestone`). Zero client-side wallet mutation.
- **Property tests with fast-check** verify the zero-sum ledger invariant across **10,000 random transition sequences** — exactly the senior-eng signal the roadmap flagged as #1.
- New migration `202604300001_add_auto_release_and_idempotency.sql`: `auto_release_milestones()` SQL function, `idempotency_keys` table with RLS, milestone-status notification trigger.
- Edge Function `supabase/functions/auto-release/index.ts` runs every 5 min, skips disputed milestones.
- Realtime subscriptions on both client and worker milestone UIs — instant status updates across both screens.
- ☒ `pnpm typecheck` 0 errors • ☒ `pnpm lint` 0 errors • ☒ `pnpm test` 13/13 passed.

Deferred (tracked for later phases):

- Mock wallet top-up screen — punted to Phase 4 polish or Phase 5.

- Mock worker withdrawal flow — same.
- Proof upload UI (photo + video + geo) — Phase 5.
- Web Push via VAPID + `send-push` Edge Function — Phase 6 (notification rows are already inserted by trigger; only the push delivery layer is missing).
- Sentry capture inside escrow Server Actions — currently `// TODO` comments, wire up in Phase 7.
- Admin `split` resolution SQL function — Phase 5 admin panel (only `force-release` and `refund` exist today).

Lessons learned:

- **Antigravity executed Step 0 housekeeping autonomously** when `AGENTS.md` was loaded — created `193b1f3` without being told. `AGENTS.md` as a project contract works.
- **Pure in-memory state-machine simulator** (`src/lib/escrow-machine.ts`) was the right call for tests — keeps Vitest deterministic and Supabase-free, and fast-check can hammer it 10k times in <2s.
- `Next.js redirect()` **throws inside try/catch** — caught during typecheck; the placeholder `fund/page.tsx` had to be reshaped to handle the throw correctly.

 **Live smoke — Apr 30, 2026, ≈02:42 IST: Phase 4 happy path GREEN end-to-end on prod** (`mypakka.vercel.app`).

Full state machine exercised across two browsers/two roles in one session:

1. **Fund (client → escrow):** Aarav (`+919876500001`) accepted Ravi's application on Drainage repair (`094b2856-19a8-498e-bd12-242557f20223`). Acceptance redirected to `/client/jobs/[id]/fund`, milestone 1 (₹15k) flipped `pending → funded`, badge `Locked In Escrow`. Wallet LOCKED ₹40k → ₹55k (+₹15k); ledger entry `Locked In Escrow -₹15,000` appended.
2. **Submit (worker → review):** Ravi (`+919876500011`) navigated `/worker/applications` (URL typed manually — nav gap logged) → job detail → `View Milestones & Submit Work` → `Submit for Review`. Milestone flipped `funded → submitted`, badge `Awaiting Review`, copy `Submitted just now. Auto-releases if client takes no action.` (`auto_release_at = +72h`). Toast displayed.
3. **Approve (client → release):** Aarav clicked Approve on the refreshed milestones page. Milestone flipped `submitted → released`, badge `Released` (emerald), copy `Released to Worker · just now`. Wallet LOCKED ₹55k → ₹40k (−₹15k debited correctly); Locked Breakdown self-corrected (drainage row vanished, only seeded `2BHK kitchen renovation ₹60,000` remains — confirming the per-job rollup recomputes on read for transitions, only the seeded kitchen drift persists). Ledger entry `Diagnose & repair drainage · Released · -₹15,000` appended.

Verifications: every transition went through `fund_escrow` / `submit_milestone` / `approve_milestone` SECURITY DEFINER RPCs (no client-side wallet writes), wallet balances are internally consistent across the transition pair (LOCKED change exactly equals milestone amount), ledger is append-only and chronological. Property tests already cover the invariant; live smoke confirms it on prod.

Bugs surfaced & logged same turn (every one in Phase 7 followups):

- Application count rendered `(0)` on `/client/jobs/[id]` despite live data → ROOT CAUSE FK-hint embed bug, fix shipped, ADR-0032/0033 .
- Worker name + trust tier render as placeholders (`Worker` / `Bronze`) → RLS-empty client-session lookup, ADR-0034 .
- Wallet LOCKED card vs breakdown drift ₹20k → seed inconsistency, ADR-0035 .
- Realtime channel does NOT deliver milestone UPDATES to open tabs → 4 ranked hypotheses, ADR-0037 .
- Worker has no nav link to `/worker/applications` → orphan URL UX bug, ADR-0036 .

What's NOT yet smoked (still open Phase 4 / 4-polish boxes):

- Mock client wallet top-up screen — Aarav's wallet was seed-funded; never tested user-initiated top-up.
- Worker withdrawable balance + mock withdrawal — Ravi's `available` should now reflect +₹15,000 from the release; not yet visually verified on `/worker/wallet`.
- Dispute path (`submitted → disputed`) — fast-check covers it in unit tests; no live UI smoke yet.
- Auto-release Edge Function in prod — cron not yet deployed (`supabase/functions/auto-release/index.ts` exists locally but `supabase functions deploy auto-release + \*/5 \* \* \* \*` schedule still pending).

- Web Push delivery via VAPID — Phase 6.
- Proof upload (photo + video + geo) — Phase 5.

Next-turn smoke targets: (1) flip to Ravi, open `/worker/wallet`, confirm `available\_balance` shows +₹15,000 and a corresponding `release` ledger row credit; (2) deploy `auto-release` Edge Function + register cron in Supabase; (3) capture origin/main HEAD + Vercel deploy ID for Phase 4 baseline.

 **Smoke #2 + final wallet verification — Apr 30, 2026 ≈14:59 IST: Phase 4 happy path GREEN END-TO-END on a SECOND independent job (2b86658c-1277-4ea7-8760-d56e9cf4a89f Real Time Smoke Test, ₹1,000).** Full state machine exercised again across two browsers / two roles, with the worker wallet credit now finally verified.

1. **Post (client):** Aarav posted *Real Time Smoke Test* (Welding category — see ⚠ below — ₹1,000 budget, 1 × ₹1,000 milestone, Hyderabad/Maisammaguda).

2. **Apply (worker):** Ravi applied ₹1,000 / 1d ETA. Discovered category-filter friction: `/worker/feed` defaults to filtering by Ravi's KYC categories (Plumbing), so Welding job was hidden until he cleared the filter — correct product behaviour, but worth a sticky banner “Showing 6 of 8 jobs · clear filter to see all” for Phase 6.6.

3. **Accept (client):** Aarav clicked Accept on the application card. Milestone 1 flipped `pending → funded`, wallet LOCKED ₹40k → ₹41k (+₹1,000).

4. **Submit (worker):** Ravi was redirected to `/worker/feed` post-OTP-login (third ADR-0036 hit in one smoke run — assigned smoke job is invisible there because feed shows `status='open'` only). Manually navigated to `/worker/applications` → job detail → *Submit for Review*. Milestone flipped `funded → submitted`. **Tab A (open) did not flip — first realtime miss in this smoke.**

5. **Approve (client):** Aarav hard-refreshed Tab A → badge correctly showed `Awaiting Review` + Approve/Dispute CTAs (DB layer fine). Clicked Approve → confirm modal *Approve & Release* → milestone flipped `submitted → released`. Tab A shows `Released` emerald + *Released to Worker · just now*. Wallet LOCKED ₹41k → ₹40k (– ₹1,000). **Tab B (open) did not flip — second realtime miss.**

6. **Worker wallet credit (the receipt):** Ravi hard-refreshed Tab B's `/worker/jobs/.../milestones` → **AVAILABLE ₹1,51,000** and **EARNED (THIS JOB) ₹1,000**. Math reconciles perfectly: ₹1,35,000 seed + ₹15,000 (drainage release) + ₹1,000 (smoke release) = ₹1,51,000 ✅. Append-only ledger holds across both transitions.

Verdict: the financial core is bulletproof. State machine, ledger atomicity, RLS isolation, SECURITY DEFINER RPC routing, wallet reconciliation — all confirmed live across two completely independent jobs. Property tests already covered the invariant; live smoke now confirms it on prod twice.

Realtime is the only red light, and Hypothesis 1 is dead. Migration `202604300002\_realtime\_replica\_identity.sql` (REPLICA IDENTITY FULL on `milestones` / `wallets` / `escrow\_ledger`, shipped in commit `a3560f1`) was a NECESSARY but NOT SUFFICIENT fix. The bug persists in BOTH directions: worker submit → client tab missed UPDATE; client approve → worker tab missed UPDATE. Same behaviour on both sides means the bug is at the **client/JS layer**, not the publication config.

Next commit ships H2 + H3 + H4 in one shot (combined fix, single PR for CodeRabbit):

- **H2 — `setAuth`:** call `supabase.realtime.setAuth(session.access\_token)` immediately after `signInWithOtp` / `verifyOtp` resolves AND on every auth state change. Without this, the channel subscribes with the anon key and Realtime evaluates the SELECT policy against the wrong JWT — payload silently dropped before reaching the client. Patch in `src/lib/supabase/client.ts`.

- **H3 — Filter syntax:** confirm both milestones pages pass `filter: 'job\_id=eq.<uuid>'` (string) NOT `filter: { job\_id: jobId }` (object). The object form silently no-ops. Patch in `src/app/client/jobs/[id]/milestones/page.tsx` and `src/app/worker/jobs/[id]/milestones/page.tsx`.

- **H4 — Channel cleanup race:** replace `channel.unsubscribe()` with `supabase.removeChannel(channel)` in the `useEffect` cleanup, and add `channel.subscribe((status) => console.log('\[milestones]', status))` so we can verify status reaches `SUBSCRIBED` in DevTools. Same two files.

- **Extended REPLICA IDENTITY migration** `202605XXXXX\_realtime\_replica\_identity\_apps.sql` adds `job\_applications` + `jobs` (still missed in `202604300002`) and audits `notifications` + `disputes` for any future subscription needs.

Smoke run #3 protocol (after the combined fix ships): post a fresh job, apply, accept, submit, approve —

every transition must flip the open opposite-tab WITHOUT a hard refresh. Also verify: (a) `/worker/jobs` no longer 404s (redirects to `/worker/applications`), (b) sticky bottom nav shell visible on every `/worker/*` and `/client/*` route, (c) worker name + trust tier render correctly on `/client/jobs/[id]` application cards (no more `Worker` / `Bronze` placeholders). If green: mark ADR-0036, ADR-0034, and ADR-0037  closed and all three Realtime / nav / RPC lines in Phase 7 followups flip green. Then move to Phase 5.

 **Combined next-commit prompt queued — Apr 30, 2026 ≈04:54 PM IST.** Single Antigravity + Sonnet 4.6 session, single PR for CodeRabbit, single smoke run #3 verification. Covers ADR-0037 (Realtime H2/H3/H4 + extended REPLICA IDENTITY migration) + ADR-0036 (nav shell + `/worker/jobs` 404 redirect + back-button audit) + ADR-0034 (`get\application\worker\summary` RPC for worker name + trust tier). Full prompt text lives in the chat thread; copy-paste into Antigravity, switch model picker to Sonnet 4.6, accept after diff review.

- Full state machine end-to-end: fund → submit → approve → dispute → auto-release, every transition via existing **SECURITY DEFINER RPCs** (`fund\escrow` , `submit\milestone` , `approve\milestone` , `dispute\milestone`). Zero client-side wallet mutation.
 - **Property tests with fast-check** verify the zero-sum ledger invariant across **10,000 random transition sequences** — exactly the senior-eng signal the roadmap flagged as #1.
 - New migration `202604300001\_add\_auto\_release\_and\_idempotency.sql` : `auto\release\milestones()` SQL function, `idempotency\_keys` table with RLS, milestone-status notification trigger.
 - Edge Function `supabase/functions/auto-release/index.ts` runs every 5 min, skips disputed milestones.
 - Realtime subscriptions on both client and worker milestone UIs — instant status updates across both screens.
 -  `pnpm typecheck` 0 errors •  `pnpm lint` 0 errors •  `pnpm test` 13/13 passed.
 - Mock wallet top-up screen — punted to Phase 4 polish or Phase 5.
 - Mock worker withdrawal flow — same.
 - Proof upload UI (photo + video + geo) — Phase 5.
 - Web Push via VAPID + `send-push` Edge Function — Phase 6 (notification rows are already inserted by trigger; only the push delivery layer is missing).
 - Sentry capture inside escrow Server Actions — currently `// TODO` comments, wire up in Phase 7.
 - Admin `split` resolution SQL function — Phase 5 admin panel (only `force-release` and `refund` exist today).
 - **Antigravity executed Step 0 housekeeping autonomously** when `AGENTS.md` was loaded — created `193b1f3` without being told. `AGENTS.md` as a project contract works.
 - **Pure in-memory state-machine simulator** (`src/lib/escrow-machine.ts`) was the right call for tests — keeps Vitest deterministic and Supabase-free, and fast-check can hammer it 10k times in <2s.
 - `Next.js redirect()` **throws inside try/catch** — caught during typecheck; the placeholder `fund/page.tsx` had to be reshaped to handle the throw correctly.
1. **Fund (client → escrow):** Aarav (`+919876500001`) accepted Ravi's application on Drainage repair (`094b2856-19a8-498e-bd12-242557f20223`). Acceptance redirected to `/client/jobs/[id]/fund` , milestone 1 (₹15k) flipped `pending → funded` , badge `Locked In Escrow` . Wallet LOCKED ₹40k → ₹55k (+₹15k); ledger entry `Locked In Escrow -₹15,000` appended.
 2. **Submit (worker → review):** Ravi (`+919876500011`) navigated `/worker/applications` (URL typed manually — nav gap logged) → job detail → `View Milestones \& Submit Work` → `Submit for Review` . Milestone flipped `funded →`

submitted, badge Awaiting Review, copy Submitted just now. Auto-releases if client takes no action. (auto_release_at = +72h). Toast displayed.

3. **3. Approve (client → release):** Aarav clicked Approve on the refreshed milestones page. Milestone flipped submitted → released, badge Released (emerald), copy Released to Worker · just now. Wallet LOCKED ₹55k → ₹40k (–₹15k debited correctly); Locked Breakdown self-corrected (drainage row vanished, only seeded 2BHK kitchen renovation ₹60,000 remains — confirming the per-job rollout recomputes on read for transitions, only the seeded kitchen drift persists). Ledger entry Diagnose \& repair drainage · Released · –₹15,000 appended.
 - • Application count rendered (0) on /client/jobs/[id] despite live data → ROOT CAUSE FK-hint embed bug, fix shipped, ADR-0032/0033 ✓.
 - • Worker name + trust tier render as placeholders (Worker / Bronze) → RLS-empty client-session lookup, ADR-0034 ⌚.
 - • Wallet LOCKED card vs breakdown drift ₹20k → seed inconsistency, ADR-0035 ⌚.
 - • Realtime channel does NOT deliver milestone UPDATES to open tabs → 4 ranked hypotheses, ADR-0037 ⌚.
 - • Worker has no nav link to /worker/applications → orphan URL UX bug, ADR-0036 ⌚.
 - • Mock client wallet top-up screen — Aarav's wallet was seed-funded; never tested user-initiated top-up.
 - • Worker withdrawable balance + mock withdrawal — Ravi's available should now reflect +₹15,000 from the release; not yet visually verified on /worker/wallet.
 - • Dispute path (submitted → disputed) — fast-check covers it in unit tests; no live UI smoke yet.
 - • Auto-release Edge Function in prod — cron not yet deployed (supabase/functions/auto-release/index.ts exists locally but supabase functions deploy auto-release + */5 * * * * schedule still pending).
 - • Web Push delivery via VAPID — Phase 6.
 - • Proof upload (photo + video + geo) — Phase 5.
1. **1. Post (client):** Aarav posted *Real Time Smoke Test* (Welding category — see ⚠ below — ₹1,000 budget, 1 × ₹1,000 milestone, Hyderabad/Maisammaguda).
2. **2. Apply (worker):** Ravi applied ₹1,000 / 1d ETA. Discovered category-filter friction: /worker/feed defaults to filtering by Ravi's KYC categories (Plumbing), so Welding job was hidden until he cleared the filter — correct product behaviour, but worth a sticky banner “Showing 6 of 8 jobs · clear filter to see all” for Phase 6.6.
3. **3. Accept (client):** Aarav clicked Accept on the application card. Milestone 1 flipped pending → funded, wallet LOCKED ₹40k → ₹41k (+₹1,000).
4. **4. Submit (worker):** Ravi was redirected to /worker/feed post-OTP-login (third ADR-0036 hit in one smoke run — assigned smoke job is invisible there because feed shows status='open' only). Manually navigated to /worker/applications → job detail → Submit for Review. Milestone flipped funded → submitted. **Tab A (open) did not flip — first realtime miss in this smoke.**
5. **5. Approve (client):** Aarav hard-refreshed Tab A → badge correctly showed Awaiting Review + Approve/Dispute CTAs (DB layer fine). Clicked Approve → confirm modal Approve & Release → milestone flipped submitted → released. Tab A shows Released emerald + Released to Worker · just now. Wallet LOCKED ₹41k → ₹40k (–₹1,000). **Tab B (open) did not flip — second realtime miss.**

6. **Worker wallet credit (the receipt)**: Ravi hard-refreshed Tab B's `/worker/jobs/.../milestones` → **AVAILABLE ₹1,51,000** and **EARNED (THIS JOB) ₹1,000**. Math reconciles perfectly: ₹1,35,000 seed + ₹15,000 (drainage release) + ₹1,000 (smoke release) = ₹1,51,000 ✓. Append-only ledger holds across both transitions.
- **H2 — `setAuth`**: call `supabase.realtime.setAuth(session.access_token)` immediately after `signInWithOtp / verifyOtp` resolves AND on every auth state change. Without this, the channel subscribes with the anon key and Realtime evaluates the SELECT policy against the wrong JWT — payload silently dropped before reaching the client. Patch in `src/lib/supabase/client.ts`.
 - **H3 — Filter syntax**: confirm both milestones pages pass `filter: 'job_id=eq.<uuid>'` (string) NOT `filter: { job_id: jobId }` (object). The object form silently no-ops. Patch in `src/app/client/jobs/[id]/milestones/page.tsx` and `src/app/worker/jobs/[id]/milestones/page.tsx`.
 - **H4 — Channel cleanup race**: replace `channel.unsubscribe()` with `supabase.removeChannel(channel)` in the `useEffect` cleanup, and add `channel.subscribe((status) => console.log('[milestones]', status))` so we can verify status reaches `SUBSCRIBED` in DevTools. Same two files.
 - **Extended REPLICA IDENTITY migration** `202605XXXXX\realtime\replica\identity\apps.sql` adds `job_applications` + `jobs` (still missed in `202604300002`) and audits `notifications` + `disputes` for any future subscription needs.

Phase 5 — Materials, disputes, admin (Day 13-16)

- ☐ Direct-to-vendor material purchase flow (mock vendor list)
- ☐ Dispute filing with evidence locker
- ☐ Admin panel (separate `/admin` route, super-admin role) for kill-switch
- ☐ Admin can refund, force-release, ban users

Phase 5.5 — Targeted AI features (Day 16-19) ★ Differentiator



Why this phase exists. Pakka uses AI as **build tooling** everywhere (Antigravity, Trae, Opus 4.6) but the **product itself** ships zero AI features today. That's a missed opportunity — AI built for the *user*, not just the developer, is what 2026 marketplaces look like. This phase adds 3 surgical, high-leverage AI features that solve real friction for both clients AND workers, without putting money at risk.

Design rules (non-negotiable):

- AI is **advisory only** for any decision involving money, trust tiers, KYC, or disputes. Humans always click the final button.
- Every AI surface has a **fallback** — the app must work end-to-end with `AI_FEATURES=false`.
- LLM calls go through **one server-side wrapper** (`src/lib/ai/llm.ts`) with timeout, retry, fallback model, and Sentry capture. Never call AI APIs from the browser.
- All prompts are versioned in `src/lib/ai/prompts/` so changes are reviewable in PRs.
- AI is **advisory only** for any decision involving money, trust tiers, KYC, or disputes. Humans always click the final button.
- Every AI surface has a **fallback** — the app must work end-to-end with `AI_FEATURES=false`.
- LLM calls go through **one server-side wrapper** (`src/lib/ai/llm.ts`) with timeout, retry, fallback model, and Sentry capture. Never call AI APIs from the browser.
- All prompts are versioned in `src/lib/ai/prompts/` so changes are reviewable in PRs.

Where AI actually moves the needle

Friction	Who feels it	AI solution	Productivity gain
Breaking a job into milestones with realistic budgets is hard	Client	AI Job Drafter — natural-language → structured job + milestones + materials	30-min job posting → 90 seconds
Workers (often not English-fluent) struggle to write compelling bids	Worker	AI Application Drafter — voice/text in any language → polished bid	Win rate ↑ + accessibility for non-literate users
Admin spends 15+ min reading dispute thread + proofs + messages to make a call	Admin	AI Dispute Summarizer — events + evidence → 1-paragraph TL;DR + recommended verdict	Triage time ↓ 80%, fewer wrong rulings
Client doesn't know which applicant to pick	Client	Smart Applicant Brief — 1-line LLM summary of each worker's track record + why-pick / why-skip	Decision time ↓, better matches
Worker submits blurry/incomplete proof → dispute risk	Worker (and client downstream)	Proof Coach — multimodal model checks photo before submit, asks for retake	Disputes ↓, auto-release rate ↑

Tier 1 — Ship in Phase 5.5 (must-haves)

☐ **AI Job Drafter** (`/client/jobs/new` → new "Describe your problem" tab)

- Client types or speaks: *"My kitchen sink has been leaking for 3 days, water pooling under cabinet, faucet also drips."*
- Server Action calls Sonnet 4.6 with structured-output schema (Zod → JSON Schema). Returns: title, category, milestones with amounts summing to a realistic budget for the city, materials list, ETA estimate.
- UI pre-fills the entire multi-step form. Client can edit any field before submit.
- **Guardrail:** budget suggestions cite recent comparable jobs from the `jobs` table (lightweight RAG over the last 200 completed jobs in same category + city). Never invent numbers.
- ~4 hrs to build. **The single best demo moment in the entire app.**

☐ **AI Application Drafter** (`/worker/jobs/[id]` → "Help me write this" button on Apply modal)

- Worker can type **or speak in Hindi / Telugu / Tamil / English** (Web Speech API → server transcription via Whisper-class model).
- LLM drafts: bid amount (anchored to job budget + worker's past win rate), ETA (anchored to past similar jobs), persuasive 2-3 sentence message highlighting relevant `worker_profiles.skill_tags` + `jobs_completed`.
- Worker reviews + edits before submit.
- **Guardrail:** bid stays within the ±20% Zod rule already enforced server-side. Message uses worker's actual stats — not free-form invention.
- ~4 hrs. **Massive accessibility win** for the target user base.

☐ **AI Dispute Summarizer** (`/admin/disputes/[id]`)

- Admin opens a dispute. AI panel auto-generates: chronological event summary, evidence inventory (photos / messages / status changes), each side's strongest argument in 1 sentence, recommended

verdict (`refund\_client` / `release\_worker` / `split 50-50`) with confidence score and reasoning.

- Admin always clicks the final button. AI verdict is shown alongside, never auto-applied.
- Uses **Opus 4.6** (1M context holds full job history + all messages + all proofs).
- **Guardrail**: prompt explicitly forbids referring to anything outside supplied context. Sentry captures every case where admin overrides the AI verdict — that's the training signal for future prompt iteration.
- ~3 hrs. **Strongest senior-eng signal of the three** — RAG, structured output, human-in-the-loop, override telemetry.

Tier 2 — Ship if time permits (nice-to-haves)

☐ **Smart Applicant Brief** — on `/client/jobs/[id]` application card, replace bare "Worker" / trust badge with a 1-line LLM-generated summary: *"12 drainage jobs (4.8★ avg), Gold tier, lives 6km away. Bid is 8% above your budget but ETA is fastest of 3 applicants."* Short, scannable, decision-ready. ~2 hrs.

☐ **Proof Coach** — on worker proof upload, multimodal model (Gemini 3 Pro Vision) checks lighting / focus / context / scope match. Returns approval-rate estimate + retake suggestions. Worker can submit anyway — coach is advisory. ~3 hrs.

☐ **Photo-to-Job** — alternative Job Drafter entry: client uploads photo of broken thing, multimodal model writes the job description. Best for non-tech-savvy clients. ~2 hrs (small extension of Job Drafter).

☐ **Weekly AI Digest Email** — Edge Function cron sends each user a personalized weekly summary: jobs/earnings/tier progress for workers; spend/active jobs/saved time for clients. LLM writes the prose, structured data drives the numbers. ~2 hrs.

Tier 3 — Already planned elsewhere

- ***kapa.ai docs chat widget*** — already in Tier 8 of the AI tool stack. Recruiters chat with `/docs/`. Ships in Phase 8 alongside Mintlify.

What we explicitly DO NOT build (and why)

Tempting feature	Why we skip
AI auto-approves proof	Money on the line. A false-approve costs the client real funds. Always human-in-the-loop for transitions out of <code>submitted</code> .
AI auto-upgrades trust tiers	Workers will reverse-engineer the prompt and game it within a week. Trust tier stays rule-based + admin-reviewed.
Generic in-app chatbot	Distracts from the escrow story. Recruiters see "yet another GPT wrapper." Skip.
AI-generated KYC verdicts	Regulatory minefield + identity-fraud risk. Admin reviews KYC manually.
LLM-written dispute resolutions as final verdicts	Liability. AI summarizes + recommends; admin decides + signs.

Tech approach

- **Single server-side wrapper** `src/lib/ai/llm.ts` exporting `complete()`, `completeWithSchema()`, `transcribeAudio()`, `analyzeImage()`. Every feature calls through this — no scattered `fetch('https://api.anthropic.com/...')` calls.
- **Models**: Sonnet 4.6 default (cheap, fast, structured-output reliable). Opus 4.6 for Dispute Summarizer (complex reasoning over long context). Gemini 3 Pro Vision for proof analysis (cheapest multimodal). Whisper API for voice transcription.

- **Provider: OpenRouter** to keep switching cheap and avoid vendor lock-in. ~₹500-1500 / month at portfolio-demo scale; ₹0 if Antigravity preview credits cover it.
- **Schema-validated outputs:** every LLM response passes through Zod. Invalid → retry once with stricter prompt → fallback to non-AI path.
- **RAG without a vector DB:** SQL filter + `LIMIT 50` + LLM summarization. We have <10k jobs at demo scale — no need for pgvector yet. Document the upgrade path in an ADR.
- **Observability:** every AI call logs prompt hash, model, latency, cost, and user verdict-override into an `ai_calls` table. PostHog dashboards show feature usage + override rate.
- **Feature flags:** `AI_JOB_DRAFTER_ENABLED`, `AI_APPLICATION_DRAFTER_ENABLED`, `AI_DISPUTE_SUMMARY_ENABLED` — independent kill switches per feature.

Resume positioning lift

Adds two lines to the resume bullet:

Shipped 3 production LLM features (Job Drafter, Application Drafter, Dispute Summarizer) with structured outputs (Zod), lightweight RAG over Postgres, multilingual voice input, and admin-override telemetry. Single-wrapper architecture (`src/lib/ai/llm.ts`) keeps prompts versioned, providers swappable, and costs observable.

Plus 2 ADRs queued for Phase 8:

- **ADR-00XX:** AI is advisory; humans always click the money button.
- **ADR-00XX:** One LLM wrapper, structured outputs, no client-side AI calls.

Phase 6 — Polish, PWA, push notifications (Day 17-20)

- ☐ PWA manifest + service worker
- ☐ Install prompt UX
- ☐ Web Push setup with VAPID keys
- ☐ Empty states, loading skeletons, error boundaries
- ☐ Hindi + English language toggle (i18n)
- ☐ Accessibility pass (keyboard nav, ARIA, contrast)

Phase 6.5 — Login moment + Personalized greeting (Day 19-20) ★ The details that separate a product from a CRUD demo



Why this phase exists. “Enter your phone number” and “Welcome back” and a giant page header that just says “Dashboard” are what every Bolt-generated marketplace ships. Pakka is not that. The login screen and the first second after login are where users decide *“this is a real product”* vs *“this is someone’s college project”* — and that judgment never gets reversed. This phase ships the small, opinionated details that make Pakka feel like Stripe / Linear / Cred and not a tutorial output. Total cost: ~6 hours. Visible payoff: every demo screenshot suddenly feels finished.

Three design rules (non-negotiable):

- Every line of copy does ONE of: build **trust**, signal **liveness**, or push toward **purpose**. If it does none, cut it.
- No stock illustrations of people staring at phones. No bare “Welcome.” No “Login” as a heading.
- All copy lives in `src/lib/copy/` — reviewable in PRs, A/B-testable, i18n-ready (en + hi from day one).

- Every line of copy does ONE of: build **trust**, signal **liveness**, or push toward **purpose**. If it does none, cut it.
- No stock illustrations of people staring at phones. No bare "Welcome." No "Login" as a heading.
- All copy lives in `src/lib/copy/` — reviewable in PRs, A/B-testable, i18n-ready (en + hi from day one).

A. The login screen — make the system feel alive before the user types anything

Five layered ideas. Ship #1, #3, #4, #5 in Phase 6.5. #2 is a stretch goal for Phase 8 demo polish.

1. **Live escrow ticker (the headline idea).** Top of `/login`, a single line fed by Supabase Realtime: *"₹4,28,000 held safely in escrow right now · 12 workers paid this week · 0 disputes pending."* Numbers tick up as users elsewhere on the platform fund / submit / approve. **You can literally watch the platform work while you sit on the login screen.** Most apps show static marketing copy here — Pakka shows the system itself. Single biggest *"wait, is this real?"* moment a recruiter will hit on the demo. Built on a `recent_activity_view` (refreshed every 60s) plus a Realtime channel that increments visible counters on insert; falls back to last-known static value if the channel disconnects.
2. **Rotating proof-of-work tiles (stretch — Phase 8 polish).** 3 cards beneath the ticker, anonymized to first name + city: *"Ramesh in Hyderabad got paid ₹15,000 for drainage repair · 4m ago"*, *"Priya's kitchen renovation hit milestone 2 of 4 · 18m ago"*, *"Auto-release moved ₹8,200 to Suresh while he was at lunch · 1h ago"*. Rotates every 5s. Fed by a `public_recent_releases` view that strips PII. The exact pattern Stripe / Vercel / Resend use on their landing pages — but for marketplace activity instead of customer logos.
3. **Two-door entry instead of one phone field.** Don't ask *"what's your phone number"* first. Ask **"why are you here"** first. Two tiles, each pre-selecting the role on the next step and addressing the actual fear of that side (client: *will I get scammed?* · worker: *will they actually pay?*):
 -  **I want to hire** — *"Find trusted workers nearby. Your money is locked safely until the job is actually done."*
 -  **I want to earn** — *"Get paid the moment you finish. No chasing clients, no broken promises."*
4. **Animated rotating verb in the headline.** Big text above the door: *"Money that funds → locks → protects → releases the work."* — one word at a time, slide-up animation every 1.2s, infinite loop. Powered by Motion. The 4 verbs *are* the escrow state machine — the headline teaches the product before the user has logged in.
5. **Footer trust strip + Demo Mode banner.** Single low-key row, always visible:  256-bit encryption ·  Milestone-based escrow ·  Built for India, in India ·  Open-source · MIT. The Open-source · MIT chip links to the GitHub repo — **recruiter magnet**, they click it, they're already on the artifact. Above the form, a Demo Mode banner: *"👋 Demo mode is on. Use +919876500001 with OTP 123456 to log in as Aarav (client), or +919876500011 for Ravi (worker)."* No "email me for credentials," no friction.

Out of scope for Phase 6.5 (deliberately): stock-photo hero illustrations, autoplay video backgrounds, animations that push Lighthouse Performance below 90, copy in fake hand-written fonts. We are *not* shipping a Lovable landing page here.

B. The personalized greeting — first 3 seconds after login decide whether the user feels seen

Replaces the hardcoded "Dashboard" header on `/client`, `/worker`, `/admin` (queued as ADR-0038, now superseded by **ADR-0039** below).

Structure of every greeting:

[time-band greeting], [first name] [emoji]
[role-specific punchy line tied to Pakka's purpose]
[data-driven status subtitle]
"[rotating quote of the day related to escrow / trust / work]"

Time bands (Asia/Kolkata for now; swap to `profiles.timezone` later):

Hour (local)	Band	Energy
04:00 – 11:59	Morning	Fresh, optimistic — let's start
12:00 – 16:59	Afternoon	Mid-day — still time to ship
17:00 – 20:59	Evening	Wrap-up — see what landed today
21:00 – 03:59	Night	Reassuring — the system has your back

Copy library — CLIENT (someone hiring):

- Morning 🌞
 - "Good morning, Aarav. Let's get your home ready before the day heats up."
 - "Rise and shine, Aarav. Three of yesterday's milestones are ready for review."
 - "Morning, Aarav. Trusted workers near you are already online."
- Afternoon
 - "Good afternoon, Aarav. Half the day's still yours — what's getting fixed today?"
 - "Afternoon, Aarav 🖐️ — your escrow is locked, your worker is on the job."
 - "Aarav, your money is safe. Now let's see if the work is too."
- Evening 🌆
 - "Evening, Aarav. Time to see how today's work stacked up."
 - "Good evening — your milestones don't sleep, but you can."
- Night 🌙
 - "Burning the midnight oil, Aarav? Auto-release has your back."
 - "Late-night planning? Post a job now, workers see it first thing tomorrow."

Copy library — WORKER (someone earning):

- Morning 🌞
 - "Good morning, Ravi. 12 new jobs near you since last night."
 - "Rise and grind, Ravi. Today's earnings are one milestone away."
 - "Morning, Ravi. Yesterday's submission auto-releases in 8 hours."
- Afternoon 🛠️
 - "Good afternoon, Ravi. Photo + receipt = paid by sundown."
 - "Afternoon, Ravi — your last job pushed your trust tier closer to Gold."
- Evening 🌆
 - "Good evening, Ravi. Submit before 9pm, sleep paid."

- *"Evening, Ravi. ₹15,000 hit your wallet today — take the win."*
- Night 🌙
 - *"Late night, Ravi? Your bid is live, the client's reading it now."*
 - *"Resting, Ravi? Good. The escrow is working while you sleep."*

Copy library — ADMIN (one line, any band):

- *"[Greeting], Sohel. 3 disputes, 2 KYCs, ₹4.2L locked. The room is yours."*

Status subtitle (data-driven, one line under the greeting — also kills the orphan-URL problem from ADR-0036):

- Client → *"X active job(s) · ₹Y locked in escrow · Z worker(s) ready for review"*
- Worker → *"X active application(s) · ₹Y on the way · Trust tier: Gold"*
- Admin → *"X disputes · Y pending KYCs · ₹Z held across the platform"*

Quote rotator — Pakka's house quotes (none of them generic):

- *"Trust is earned in milestones, not promises."*
- *"Cash on completion is yesterday. Escrow on milestones is forever."*
- *"A handshake doesn't scale. A ledger does."*
- *"The best protection is transparency. The second-best is auto-release."*
- *"Workers don't want sympathy. They want to get paid on time."*
- *"Disputes shouldn't be a fight. They should be a function call."*
- *"Every milestone is a small contract that pays itself."*
- *"The money moves the moment the work does. Not before, not later."*

Picked deterministically from `hash(user\_id, YYYY-MM-DD) % quotes.length` — each user sees the *same* quote all day (so it feels intentional, not random) and a new one tomorrow (so it never gets stale). Hindi translations ship in the same release.

C. Implementation

• New files:

- `src/lib/copy/greetings.ts` — copy library, exports `getGreeting({ role, firstName, hour, locale })`.
- `src/lib/copy/quotes.ts` — quote library + `getDailyQuote({ userId, date })` deterministic picker.
- `src/lib/time/greeting-band.ts` — `hourToBand(hour: number): "morning" | "afternoon" | "evening" | "night"`.
- `src/components/greeting-block.tsx` — server component, takes `{ profile, stats }`, renders heading + subtitle + quote.
- `src/components/login/live-ticker.tsx` — client component, subscribes to `recent\_activity\_view` via Supabase Realtime.
- `src/components/login/role-doors.tsx` — the two-tile entry.
- `src/lib/copy/\_\_tests\_\_/greetings.test.ts` — Vitest snapshot test asserting every (role × band) has at least one line of copy. Catches missing locale strings before they ship.

• Modified files:

- `src/app/client/page.tsx`, `src/app/worker/page.tsx`, `src/app/admin/page.tsx` — replace hardcoded “Dashboard” with `<GreetingBlock />`. Pull `profiles.full_name` and role-specific stat counts server-side via the existing authenticated session — zero extra round-trip (the layout already loads the profile for the nav shell from ADR-0036).
- `src/app/login/page.tsx` — replace single phone field with `<RoleDoors />` + `<LiveTicker />` + footer trust strip + Demo Mode banner.
- **New view:** `recent_activity_view` (materialized, refreshed every 60s) — joins `escrow_ledger` + `milestones` + `profiles.first_name` + `profiles.city`, strips PII, returns last 50 public-safe events for the live ticker.

D. Resume positioning lift

Adds one bullet under the AI features bullet:

Obsessed over the small details: live escrow ticker on `/login` (Realtime-powered), role-aware time-of-day greetings, deterministic daily quote rotator, and a two-door entry replacing the generic phone-field login. Copy library is i18n-ready (en + hi) and snapshot-tested.

And one new ADR queued for Phase 8:

- **ADR-0039 (supersedes ADR-0038):** authenticated landing pages render `<GreetingBlock />` (greeting + status subtitle + daily quote), never a static “Dashboard” string. The login screen renders `<LiveTicker />` (real escrow activity), `<RoleDoors />` (purpose-first entry), and a Demo Mode banner with seeded credentials. Stock illustrations and bare “Welcome” headers are forbidden — every surface earns its space by building **trust**, signaling **liveness**, or pushing toward **purpose**.

Phase 6.6 — Wow moments at every screen (Day 20-22) ★ The senior-eng polish layer



Why this phase exists. Phase 6.5 fixed the login + dashboard. But there are 18 more screens in Pakka — and every single one is currently a generic CRUD form. Recruiters who built at Zomato / Swiggy / Urban Company / Cred have *trained eyes*: they spot the difference between a tutorial output and a real product in the first 3 seconds of any screen. This phase is the screen-by-screen audit that closes that gap. Each item below is a specific, shippable detail — none of it is decoration, all of it is product.

The rule of three (every interaction must do at least one):

- **Build trust** — show the system doing what it promised
- **Signal liveness** — prove the platform is alive right now
- **Push toward purpose** — get the user one step closer to the money moving

If an interaction does none of the three, it's removed. No decoration. No stock illustrations. No “Welcome to Pakka” headers.

Reference apps (study these, don't copy them): **Cred** for micro-interactions, **Linear** for keyboard-first power UX, **Stripe Dashboard** for liveness, **Swiggy** for empathetic empty states, **Uber** for slide-to-confirm financial actions.

- **Build trust** — show the system doing what it promised
- **Signal liveness** — prove the platform is alive right now
- **Push toward purpose** — get the user one step closer to the money moving

1. Onboarding (worker KYC) — turn 6 boring fields into a 6-step game they want to finish

- **Progress arc, not bar.** A circular gauge filling emerald clockwise. Each completed step pulses once. Bottom copy: “Step 3 of 6 · ₹0 → ₹15,000 in your first job after this.” Money is the carrot, not abstract

progress.

- **Aadhaar OCR pre-fill.** Multimodal LLM (Gemini 3 Pro Vision, already in stack for Proof Coach) reads the uploaded Aadhaar image, extracts last 4 + name. User confirms with one tap instead of typing. Drops KYC time from 4 min to 45s.
- **Selfie capture with live face guide.** A ring overlay turns gray → amber → emerald as face fits the frame. Capture button only enables when emerald. No “please retake” loops, no rejected uploads in admin queue.
- **Skill tags use chips, not multi-select.** Tap to toggle, instant visual feedback. Workers who can't read English fluently can still recognize the icon + Hindi label combination.
- **“You're verified, here's your first lead” moment.** The instant admin approves KYC, a push notification fires with one open job in the worker's category + city: *“Welcome, Ravi. ₹8,000 plumbing job 2.3km away — be the first to bid?”*

2. Onboarding (client) — show what's possible before they fill anything

- **City pulse card on the empty dashboard.** *“147 milestones completed in Hyderabad this week. Average payout: ₹6,200. Average time client → approve: 4h 12m.”* Numbers from `recent_activity_view` (the same one powering the login ticker — reuse the infra). Builds trust before the first job is even posted.
- **City auto-detected from IP,** one-tap confirm. No dropdown searching for “Hyderabad” in a 700-row list.
- **Empty job list copy:** *“No jobs yet. Most clients post their first in under 2 minutes. Start with a photo of the issue — we'll handle the rest.”* — deep links straight to the AI Job Drafter's photo entry (Phase 5.5 Tier 2).

3. Job posting (client) — replace the 6-step form with a conversation

- **Default tab is AI Job Drafter, not the manual form.** The manual form lives behind a *“Skip the AI, fill manually”* link at the bottom. 90% of clients will use the conversational entry.
- **After AI draft, show the full escrow timeline before posting.** Animated horizontal timeline: *“Today: ₹15,000 locks in escrow → Day 3: Ravi submits photo proof → Day 4: You approve, ₹15,000 releases.”* Each step has a tiny icon. The client sees the whole lifecycle before they commit a rupee. Zomato shows you the full delivery flow before you order; Pakka shows the full payout flow before you fund.
- **Smart milestone splitter.** The total budget is a horizontal bar; milestones are draggable segments. Drag the boundary, both adjacent milestones rebalance instantly. Feels like a budget game, not a form. Inline warning if any milestone is <₹500 (“too small to matter”) or >50% of total (“too much risk in one bucket”).
- **“Post job” CTA morphs.** Tap → button collapses into a lockbox icon → routes to fund screen with the lockbox icon already on screen as the visual anchor. One continuous gesture, no jarring page swap.

4. Worker browse feed — make it feel like Swiggy's restaurant feed, not LinkedIn's job feed

- **Hero card at top: live earning potential.** *“₹42,800 in open jobs near you right now · 6 in your category.”* Sums `total_budget` of open jobs within the worker's radius. Updates live via Realtime. The number is a stronger hook than any marketing copy.
- **Heat indicators on job cards.** 🔥 if posted <30min ago · 👁️ “4 viewing now” (Realtime presence channel) · ⚡ “Bid in <5m to be first” if no applications yet. Same urgency cues Swiggy uses for restaurants.
- **Pull-to-refresh: trust badge polish.** Instead of a spinner, the worker's trust-tier shield does a tiny “polish” Rive animation as new jobs load. Brand-reinforcing AND functional.

- **Skeleton loaders are job-card-shaped**, with brand-emerald shimmer (not gray). Lighthouse-friendly, instantly recognizable as “more is loading” not “the app is broken.”
- **Empty state with an action chip inline.** “No plumbing jobs within 10km right now.” — followed by tappable chips: `\[Widen to 25km · 8 jobs]` `\[Browse all categories · 23 jobs]`. The empty state IS the next step.
- **Sticky banner at top when worker has active applications/jobs.** “You have 1 active job: Drainage repair (₹15,000 · ready to submit)” → tap routes to `/worker/applications`. Solves the orphan-URL problem from ADR-0036 at the surface level.

5. Worker job detail + apply — turn the bid into a tactical move

- **Bid coach panel** (uses Phase 5.5 AI Application Drafter). “Average accepted bid for drainage in Hyderabad: ₹14,200. Your win rate at this price: ~62%. Try ₹13,800 for ~78%.” Anchored, data-driven, never invented. Worker can override.
- **Bid is a slider, not a number input.** Snaps to ₹100 increments, with the win-rate estimate updating live as the worker drags. Game feel, real data underneath.
- **CTA copy borrows the escrow language.** “Lock my bid · ₹13,800” (not “Submit application”). Mirrors the language the worker will see at every other step (lock / submit / release).
- **Confirmation animation: envelope sealing.** Rive: paper folds, trust-badge stamps as the wax seal. 1.2s. Routes to applications tab with the new application already highlighted. No toast — the animation IS the confirmation.

6. Client job detail + choosing applicant — decision-grade UX

- **Each application card uses Phase 5.5 Smart Applicant Brief.** One sentence written by Sonnet 4.6: “Ravi · 12 drainage jobs (4.8★ avg) · Gold tier · 6km away · Bid 8% above your budget but ETA fastest of 3 applicants. Why pick: highest completion-rate. Why skip: pricier than alternatives.” Decision-ready in one glance.
- **Stack-rank toggle.** Sort by: win rate · distance · bid · trust tier · ETA. Default to a weighted score. Each option re-orders the cards with a Motion staggered transition (300ms).
- **30-second undo on Accept.** The Accept Server Action queues for 30s with a sticky bottom sheet “Locking ₹15,000 with Ravi · undo (28s)”. After 30s, it commits and the auto-rejection of other applicants fires. Eliminates the “oh shit, wrong button” panic that kills trust.
- **Rejected applicants get a soft landing.** Push notification: “Drainage job went to another worker. Try this one — same category, ₹2k more, 3km closer.” Server-side query for the next-best match. Workers don’t disappear into a black hole; the platform routes them.

7. Escrow funding — money leaves the wallet, this should FEEL

- **Slide-to-confirm, not tap-to-confirm.** Like Uber’s “slide to call,” Cred’s “slide to pay.” Tactile gestures = perceived security. The slider track has a subtle gradient that brightens as the user drags. “Slide to lock ₹15,000 in escrow.”
- **Vault animation on commit.** Full-screen Rive: wallet card → padlock closes around it → routes to milestone view. 1.8s. Optional haptic vibrate via PWA Vibration API. The lockbox icon becomes a recurring brand asset everywhere escrow is mentioned (wallet, milestone cards, ledger, login ticker).
- **Receipt-style confirmation.** A small Stripe-style receipt slides up: ledger entry id, timestamp, milestone id, “funds locked.” Recruiters see this and *immediately* recognize the financial-grade thinking.

8. Milestone submit (worker) — turn proof upload into a checklist they want to complete

- **Proof Coach** (Phase 5.5 Tier 2) renders inline, not after submit. Live checklist:  photo bright enough ·  shows full scope ·  try a closer angle ·  geo-tag matches job location. Each criterion turns emerald in real-time as the worker reframes. Reduces dispute rate without auto-rejecting.
- **Submit CTA shows the auto-release countdown inline.** *"Submit for review · auto-releases in 72h if client doesn't respond · clients on this app usually approve in ~4h."* Statistic from `escrow\_ledger` aggregates. Sets correct expectations, kills anxiety.
- **Post-submit: quiet, not celebratory.** *"✓ Submitted. Sit tight — you'll know first."* with the live auto-release timer ticking visibly. Confetti is the wrong call here — this is work, not a game won.

9. Milestone approve (client) — celebrate the trust moment

- **Approve confirmation borrows the worker's effort.** *"Releasing ₹15,000 to Ravi. He's been working since 9:42am — 3h 18m on this milestone."* Empathy via data. Makes the click feel like a reward, not a transaction.
- **Wallet flip animation on commit.** Rive: client wallet ticks down ₹15,000, vault opens, money flies to worker wallet, worker wallet ticks up. Both screens animate via Realtime.
- **Worker push notification.** *"₹15,000 just hit your wallet 🎉. You earned it."* Never *"Status update: milestone released."* Every push reads like a person sent it.
- **Optional 3-tap rating after approve.** *"How did Ravi do?"* — emoji scale, single screen. Feeds the trust-tier algorithm. Skippable, never blocking.

10. Wallet pages — make the balance feel like a story

- **Animated counter on load** (Motion's `animateValue` ticking from 0 to current balance over 600ms). Subtle, professional, feels alive. Stripe Dashboard pattern.
- **Locked breakdown is tappable.** Each row is a mini job card that routes to the milestone. Long-press shows *"auto-releases on Apr 30, 8:42pm."*
- **Worker wallet has a Strava-for-paychecks panel.** *"This week: ₹38,000 · last week: ₹29,500 ·  +28%. Trust tier progress: 4 jobs to Gold."* Workers see their trajectory, not just their balance.
- **Withdrawal flow.** Pre-fills last-used UPI (Aarogya Mitra-style), one-tap confirm. Demo Mode footer: *"In production, this routes to your UPI in <8s via Razorpay Payouts."* Recruiters see the production-swap path inline.

11. Notifications — every push reads like a person sent it

- **Voice = teammate, not system.** *"Ravi just submitted milestone 1 — there's a photo waiting"* (vs *"Update: status=submitted"*). All notification copy lives in `src/lib/copy/notifications.ts` alongside the greeting library, reviewable in PRs.
- **Quiet hours by default 10pm-7am IST.** Override only for disputes (which are real emergencies). Configurable per-user in profile.
- **Daily digest at 9am IST.** Per-role: client → *"3 milestones await your review · ₹42,000 locked"* · worker → *"₹15,000 released yesterday · 4 new jobs in your category"* · admin → *"2 disputes opened overnight · 1 KYC pending."* Edge Function cron, ships in Phase 6.

- **In-app bell has a digest mode.** Tapping the bell shows today's events grouped by job, not a flat reverse-chronological list. Like Linear's inbox.

12. Empty states — every blank screen teaches what's next

Screen	Empty copy + inline action	/worker/feed (no matches)	"No plumbing jobs within 10km right now." • chips: \[Widen to 25km · 8 jobs] \[All categories · 23 jobs]
/client/jobs (first-time)	"No jobs yet. Most clients post their first in under 2 minutes — start with a photo of the issue." → AI Job Drafter	/worker/applications (no apps)	"You haven't applied to anything yet. 12 jobs in your category are waiting." → /worker/feed
/admin/disputes (zero)	"0 open disputes 🎉 — last one resolved 14h ago in 3m 22s." (operational pride, not boredom)	/worker/wallet (zero earnings)	"Your first ₹ is one milestone away." → /worker/feed
/client/wallet (zero locked)	"No money in escrow. When you accept a worker, ₹X locks here automatically until they finish."	Search no results	Suggest 3 nearest matches by category. Never a bare "No results."

13. Loading states — never a generic spinner

- **Skeleton screens shaped like the actual content.** Job cards become job-card-shaped skeletons; milestone rows become milestone-shaped. Brand emerald shimmer (200ms wave), not gray.
- **Multi-step actions get a progress narrative, not a spinner.** Escrow funding: "Locking funds → Notifying Ravi → Done." Three states, one full-screen overlay, dismisses on completion. Stripe Checkout pattern.
- **Optimistic UI everywhere it's safe.** Apply, accept, approve, dispute — UI updates instantly, server confirms in background, rolls back with a toast on failure.

14. Error states — own the failure, suggest recovery

- **Inline form validation, never generic.** "Bid must be within ±20% of ₹15,000 (₹12,000–₹18,000)." Show the exact valid range, not "Invalid amount."
- **500 / network errors get a recovery card.** "Something broke on our side. Tap to retry, or report this — we'll fix it before you blink." The Report button captures the Sentry event id and lets the user copy it.
- **Realtime disconnect banner.** When the WebSocket drops, a thin amber strip appears at the top: "Live updates paused · tap to refresh." Solves ADR-0037 at the UX layer until the channel fix ships.

15. Trust tier progression — make leveling up feel earned

- **Quiet progress arc on worker dashboard.** "4 more on-time milestones to Gold." Animated ring around the trust badge fills as jobs complete.
- **Tier-up moment.** Subtle Rive animation (badge polishes, color deepens) + push: "You're now Gold. Clients see this on every bid you make. ~12% more wins on average." Practical reward, not gamification fluff.
- **Tier-down is dignified.** No "You lost Gold!" red banner. Quiet copy: "Gold is paused — 2 on-time deliveries to restore."

16. Disputes — the most emotional moment, the calmest UX

- **Filing flow uses neutral language.** "File for review" not "Raise dispute." Reduces escalation. (Cred's complaint flow does this masterfully.)

- **Both parties see the same shared timeline view.** Transparency = trust. Each event (status change, comment, evidence upload) is a row with attribution + timestamp. No hidden information.
- **Admin AI Summarizer renders inline** (Phase 5.5). Admin's recommended verdict + reasoning visible to admin only; clients/workers only see the timeline.
- **Resolution copy is human.** *"Reviewed by Sohel · refunded ₹15,000 to Aarav · reason: photo proof did not show milestone scope."* Not *"Status: resolved_client."*

17. Admin panel — operational dashboards that look like Linear, not WordPress

- **KPI cards animate on load** (Motion staggered, 80ms between each). Linear-grade entrance.
- **Keyboard shortcuts in the dispute queue.** **J/K** to navigate, **A** to approve, **R** to refund, **S** to split, **Cmd+K** for command palette. Power-user signal.
- **Live ops feed.** Right-side panel streams recent platform events (funded / submitted / approved / disputed) — same Realtime channel as the login ticker, but full-fidelity for admins. Lets the admin spot anomalies in real-time.
- **Dispute card has the AI verdict + a confidence bar.** Admin still clicks the final button (Phase 5.5 design rule), but sees *"Refund client (87% confidence) — 4 of 5 photos missing scope"* upfront. Triage time drops from 15min to ~2min.

18. Pull-to-refresh, page transitions, micro-haptics — the connective tissue

- **Pull-to-refresh on every list.** Tiny Rive lockbox shakes off dust. Brand-reinforcing, instantly recognizable.
- **Page transitions: shared layout via Motion's `layoutId`.** Job card → job detail uses the same DOM node, animated. Feels native, not web-y. Linear and Cred both ship this.
- **Haptics on every confirm / release / error** (PWA Vibration API). Subtle: 10ms on tap, 30ms on confirm, 50ms+50ms double on error. Disabled if the device doesn't support haptics.
- **Sticky bottom action bars on mobile.** Primary CTA always within thumb reach. Never a CTA hidden above the fold.

Wow-moment tracker

#	Surface	Phase	Hours	Tier
2	Client onboarding	6.6	2	Must
4	Worker feed	6.6	4	Must
6	Client choose applicant	6.6 (uses 5.5 AI)	3	Must
8	Milestone submit	6.6 (uses 5.5 Coach)	3	Must
10	Wallet pages	6.6	3	Must
12	Empty states (×7)	6.6	3	Must
14	Error states	6.6	2	Must
16	Dispute UX	6.6 (uses 5.5 AI)	3	Must
18	Connective tissue	6.6	3	Must

Total: ~54 hours of polish work spread across Day 20–22 of Phase 6.6. Realistic with the AI tool stack already pinned (v0 for components, Rive for animations, Motion for transitions, Recraft v3 for icons).

Implementation rules

- **All copy lives in** `src/lib/copy/` (**en + hi from day one**). No hardcoded English strings in components.
- **All animations route through** `src/lib/motion/` **primitives** (`<FadeIn>`, `<SlideInFromBottom>`, `<CountUp>`, `<RiveAsset>`). Every PR adding animation must reuse these — no scattered `framer-motion` imports.
- **Every interaction is classified.** PRs that add UI without a `// trust | liveness | purpose` comment on the component get blocked at review (CodeRabbit rule).
- **Performance budget: Lighthouse Performance ≥ 90 on every page.** Animations that drop the score get cut, no exceptions.
- **Reduced-motion respected via** `prefers-reduced-motion`. All animations have a static fallback.

Resume positioning lift

Adds one bullet under the Phase 6.5 polish bullet:

Audited 18 product surfaces for senior-eng polish: slide-to-confirm escrow funding (Uber/Cred pattern), Rive money-flow animations, inline AI bid coach with live win-rate, 30-second undo on accept, deterministic confetti policy (work isn't a game), Linear-style keyboard shortcuts in admin, contextual empty states with inline action chips, real-time disconnect banner. Every surface earns its space by building **trust**, signaling **liveness**, or pushing toward **purpose**.

ADRs queued for Phase 8

- **ADR-0040: Micro-interaction taxonomy.** Every UI surface in Pakka must be classified as `trust-build`, `liveness-signal`, or `purpose-push`. Decoration is removed. PRs without a classification comment get blocked.
- **ADR-0041: Copy lives in** `src/lib/copy/`. No hardcoded user-facing strings in components. en + hi from day one. Snapshot-tested.
- **ADR-0042: Animation primitives in** `src/lib/motion/`. Every animation routes through a shared primitive with a `prefers-reduced-motion` fallback. No scattered animation library imports.

Phase 7 — Testing, observability, security (Day 21-24)

- ☐ Playwright E2E tests for: signup, post job, full escrow happy path, dispute path
- ☐ Vitest unit tests for state machine transitions (critical!)
- ☐ Sentry integration
- ☐ PostHog events on key actions
- ☐ Security: rate limiting, RLS audit, input validation with Zod
- ☐ README with architecture diagram, demo creds, video walkthrough link

Phase 8 — Ship the artifacts (Day 25-28)

The deployed app is just *one* of five artifacts. Recruiters mostly review the GitHub repo, the README, and the writeups — not the live app.

- ☐ Production deploy on Vercel with custom domain (Demo Mode banner always visible site-wide)
- ☐ Supabase prod project with seeded demo data covering every UI state
- ☐ Polish the README: architecture diagram, state machine diagram, ER diagram (all Mermaid), demo creds, embedded Loom, "fork this for your marketplace" guide

- ☐ Add `/docs/adr/` with 5-7 Architecture Decision Records (why Postgres state machine, why RLS over middleware, why PWA + TWA, why simulated money, etc.) — huge senior-engineer signal
- ☐ Add `/docs/production-swap.md` documenting how to swap mock wallet for Razorpay Route
- ☐ PWABuilder → signed AAB → Play Store internal testing track (no public release needed)
- ☐ Record 90-second Loom demo (signup → fund → submit → approve, narrated)
- ☐ Publish technical blog post on dev.to / Hashnode (pick one: "Why your escrow state machine belongs in Postgres," "RLS as your auth layer," "Atomic ledger transitions with SECURITY DEFINER + SELECT FOR UPDATE")
- ☐ LinkedIn post with architecture diagram, link to repo + demo + blog
- -
- -

The Jaw-Dropping AI Tool Stack — April 2026 Edition



Goal of this section: Every layer of Pakka — code, UI, animation, icons, copy, demo video — should feel like a senior team built it, not a solo undergrad. This is the tool-by-tool breakdown to get there. Pick the *Recommended* column; ignore the alternatives unless that pick breaks for you. Don't install everything at once — the *Install order* at the end of this section tells you what to add and when.

Tier 1 — Code generation & reasoning (the daily drivers)

Purpose	Recommended	Strong alternatives	Why this beats alternatives
Hard-phase IDE (escrow, security audit)	Antigravity • Claude Opus 4.6	Claude Code, Cursor + Opus 4.6	Multi-agent workflow superior for complex multi-file reasoning. Opus 4.6 leads SWE-bench Verified at 80.8% with 1M context — only model that holds the full escrow + RLS surface coherently.
Fast-phase IDE (jobs CRUD, polish)	Trae • GPT-5.3-Codex or Sonnet 4.6	Cursor, Windsurf, Zed	Free, fast, multi-model. Sonnet 4.6 hits 79.6% SWE-bench — sweet spot for routine work without burning Opus credits.
Background autonomous runs	Claude Code (terminal)	OpenAI Codex CLI	Run while you sleep — refactors, dep bumps, doc generation. Most capable agentic terminal tool in 2026.
Cost-sensitive bulk refactor	Gemini 3.1 Pro	MiniMax M2.5	80.6% SWE-bench at \$2/\$12 (vs Opus \$5/\$25) — 2.5× cheaper for 0.2 points less. Huge context for repo-wide passes.

Tier 2 — UI / Design generation

Purpose	Recommended	Strong alternatives	Use it for
Production React/Tailwind components from a prompt	v0 by Vercel	Subframe, Magic Patterns, Flowstep	Generate the dashboard cards, milestone status timeline, KYC review queue UI — paste straight into shadcn/ui.
Full-app concept exploration	Lovable (you already have Pro)	<u>Bolt.new</u> , Base44	Already paid for. Use it to draft the marketing/landing page, not the app itself.
UX audit + heatmap on existing UI	UX Pilot	Attention Insight	Run after Phase 6 polish — predictive heatmaps catch dead zones recruiters notice.

Purpose	Recommended	Strong alternatives	Use it for
Figma → polished refinement	Figma Make • Galileo AI	Stitch (Google), Moonchild	Galileo for layout exploration, Figma Make for visual polish. Free Figma tier is enough.
Design system page	Subframe	UXPin, Moonchild	If <code>/admin</code> or worker dashboards start feeling messy, Subframe exports clean React/Tailwind primitives.

Tier 3 — Animation, motion, 3D (the “wow” layer)

Purpose	Recommended	Strong alternatives	Where it goes in Pakka
Page transitions, layout animations, micro-interactions	Motion (motion.dev)	React Spring	Milestone status badge transitions, modal mount/unmount, button press feedback. MIT license, modern API, 120fps target.
Scroll-triggered hero animations	GSAP • ScrollTrigger	Motion's scroll APIs	The escrow state machine diagram on the landing page — animate funded → submitted → approved as user scrolls.
State-driven 2D micro-interactions	Rive	Lottie	Trust tier badge animations (bronze → silver → gold), the wallet “money locked” animation, success/dispute reactions.
Empty states + loading illustrations	LottieFiles	Rive	“No jobs posted yet” empty state, OTP pulse, KYC pending hourglass.
3D hero scene on landing page	Spline (web, AI prompts)	Three.js + React Three Fiber	Optional. A subtle floating shield + handshake 3D scene on the landing hero. Recruiters notice instantly.

Tier 4 — Icons, brand assets, illustrations

Purpose	Recommended	Strong alternatives
App icon (1024×1024 for PWA + Play Store)	IconikAI	SVGMaker, Recraft v3
UI icons (status, category)	Lucide (already in stack) + Recraft v3 for custom flat vector	Iconify, SVGMaker
Marketing illustrations on landing page	Recraft v3	Midjourney v7, Ideogram
Logo + wordmark	Ideogram	Recraft, Logo Diffusion
Hero / Open Graph images	Midjourney v7 or Nano Banana Pro (Gemini)	FLUX, Krea AI
Convert raster → clean SVG	Vectorizer AI	SVGMaker

Tier 5 — Code review, security, DevX

Purpose	Recommended	Where to plug in
AI PR review (catches what you miss)	CodeRabbit	GitHub App. Free for open-source. Add before Phase 4.
Codebase chat (“how does fund_escrow work?”)	Greptile or DeepWiki	Free for public repos. Embed link in README so recruiters can chat with the codebase.
Auto-fix issues to PRs	Sweep	Optional. Generates PRs from GitHub issues.

Purpose	Recommended	Where to plug in
Static security analysis	Semgrep • SonarQube Community	CI step in Phase 7. Rule-based, low noise — complements Opus's Phase 7 audit.
Dependency security	Dependabot • Socket.dev	GitHub-native. Socket catches malicious npm packages.
Large-codebase context for AI	Augment Code	Optional. Becomes useful only after Phase 5 when the repo grows.

Tier 6 — Testing automation

Purpose	Recommended	Why
E2E tests (already planned)	Playwright • AI codegen	Generates tests from natural language. Use for happy + dispute paths in Phase 7.
Visual regression	Chromatic	Free for open-source. Catches UI drift across PRs. Recruiters see a "✓ Visual tests passed" badge on every commit.
AI-driven exploratory testing	Stagehand by Browserbase	Typed browser automation. Run a daily AI smoke run in CI that explores the demo and reports anomalies.
Property-based tests for ledger math	fast-check (Vitest plugin)	Verify ledger zero-sum invariant across 10,000 random transaction sequences. #1 senior-eng signal in this entire project.

Tier 7 — Observability + analytics (already in stack, leveling up)

Purpose	Recommended	Notes
Error tracking	Sentry (already configured)	Enable Sentry AI Issue Triage in dashboard — auto-clusters errors and suggests fixes.
Product analytics + session replay	PostHog (already configured)	Enable PostHog AI insights for conversion funnel auto-analysis.
DB query observability	Supabase Logs • pg_stat_statements	Free, built in. Turn on slow query log.
Web vitals + RUM	Vercel Speed Insights	One toggle in Vercel. Bakes Lighthouse-grade RUM into every prod deploy.

Tier 8 — Documentation, demos, marketing

Purpose	Recommended	Use it for
Docs site (ADRs + Production Swap guide)	Mintlify or Nextra	Ship at <code>docs.mypakka.vercel.app</code> . Mintlify free for open-source.
Polished 90-second hero demo video	Tella or Screen Studio	Tella for layouts/zoom-pan; Screen Studio for Mac-native auto-zoom. Skip Loom for the one hero demo.
Quick async dev logs (PR descriptions)	Loom AI	Auto-summary + chapters.
Interactive clickable product demo	Arcade or Supademo	Embed on landing page so recruiters click through without installing the PWA.
AI avatar narration on demo	HeyGen	Optional. Lets you narrate the demo without recording your voice.
Diagrams (architecture, ER, state machine)	Mermaid • Excalidraw	Mermaid renders on GitHub natively. Excalidraw for the 1 hero diagram on the landing page.
AI chat assistant on your docs	kapa.ai	Optional. Lets recruiters chat with the documentation. Strong wow factor.

Tier 9 — AI for you, the developer

Purpose	Recommended
Tech research with citations	Perplexity Pro
Long-PDF reasoning (RBI guidelines, Razorpay docs)	Claude.ai (Sonnet 4.6 in chat)
Codebase Q&A on third-party libs	Greptile or DeepWiki
Model routing across all providers	OpenRouter

- —

🌟 Where each tool lights up Pakka — concrete plug-in points

This section maps each tool to a *specific moment* in the build, so you never wonder “where does this go?”

Phase 3 — Job posting + browse

- **v0**: Generate the multi-step “Post a Job” form UI. Paste output into `src/app/client/jobs/new/page.tsx`.
- **Recraft v3**: Generate the 10 category icons (masonry, plumbing, electrical, etc.) as flat SVGs in one prompt.
- **Motion**: Animate the milestone-list reorder when client adds milestones.
- **CodeRabbit**: Review the PR. Catches the “did you forget RLS on `job_applications`?” you missed.

Phase 4 — Escrow ★

- **Antigravity + Opus 4.6**: This is the phase Opus exists for. Multi-agent workflow lets one agent write `approve_milestone()`, another write the Vitest property tests, a third audit for race conditions.
- **fast-check**: Property-test that ledger sums to zero across 10,000 random sequences. Single best signal of senior engineering on the whole repo.
- **Rive**: Build the “money flying from client wallet → escrow lockbox → worker wallet” animation. **The single most viral asset you can ship.**
- **Sentry AI Issue Triage**: Catches the inevitable race condition during dev.

Phase 5 — Materials, disputes, admin

- **v0**: Generate the admin dispute queue UI with kill-switch buttons (Refund / Release / Split slider).
- **Lottie**: “No open disputes 🎉” empty state on admin dashboard.
- **Subframe**: Build the admin design system if `/admin` starts feeling messy.

Phase 6 — Polish + PWA

- **GSAP + ScrollTrigger**: Landing page hero animation — escrow state machine animates as user scrolls.
- **Spline**: Optional 3D shield+handshake hero scene.
- **IconikAI**: PWA icon set (192, 512, maskable, iOS splash screens) in one go.
- **UX Pilot**: Audit the worker-onboarding flow before locking it.
- **Chromatic**: Lock in visual regression baseline so future PRs don’t break the polish.

Phase 7 — Tests, security, observability ★

- **Antigravity + Opus 4.6:** Adversarial RLS audit. "Pretend you're a malicious worker — find every policy gap."
- **Playwright AI codegen:** Generate the happy + dispute path E2E tests from natural language.
- **Stagehand:** Daily AI smoke-test in CI on the deployed demo.
- **Semgrep:** Catch CSP, XSS, SSRF patterns automatically.
- **Socket.dev:** Verify no malicious deps slipped into the lockfile.

Phase 8 — Ship

- **Tella:** Record the 90-second hero demo. Auto-zoom on every action.
- **Mintlify:** Spin up `docs.mypakka.vercel.app` with the ADRs + production-swap guide.
- **Arcade:** Build a clickable interactive demo embedded on the landing page.
- **Excalidraw:** Final architecture diagram for the README hero.
- **HeyGen** (optional): AI-narrated voiceover on the Tella demo if you don't want to record voice.
- —

Install order — what to add today vs later



Don't install everything at once. That's the rookie move and you'll drown in tabs and subscriptions. Add tools as the phase needs them. Below is the exact order.



Zero-budget mode (this is the default for Pakka). Every tool below has a free tier that's enough for this project. **You do NOT need Anthropic Console, Claude Code, Midjourney, HeyGen, Screen Studio, or any paid subscription.** Antigravity gives free access to Sonnet 4.6 and Opus 4.6 during preview — that's all you need for the hard phases. The only money you might ever spend is the optional Play Store \$25 one-time developer fee in Phase 8, weeks away.

Today (before Phase 3 starts) — all free:

1. Install **Google Antigravity** (search "Google Antigravity" — official IDE site). Sign in with Google. The free tier includes **Sonnet 4.6**, **Opus 4.6**, and **Gemini 3 Pro** with daily quotas — plenty for this project.
2. Keep **Trae** installed as the fast-iteration IDE (also free, multi-model).
3. Add **CodeRabbit** GitHub App to the `Pakka` repo — free for open-source.
4. *(Optional, skip if you're tight on time)* Free Perplexity account for research.

Skip these — paid and not needed:

-  Anthropic Console / API top-up — Antigravity gives free Claude access
-  Claude Code terminal — nice to have, not required. Skip it.
-  Midjourney, HeyGen, Screen Studio — free alternatives below cover everything

Before Phase 4 (escrow):

1. Sign up for **Rive** — start the money-flow animation early so it's ready when the UI lands.
2. Add **fast-check** to dev deps: `pnpm add -D fast-check`.

Before Phase 6 (polish):

1. Sign up for **v0**, **Recraft v3**, **IconikAI**.
2. Install **Motion**: `pnpm add motion`.
3. Install **GSAP**: `pnpm add gsap` (free for non-commercial; Pakka qualifies as MIT open-source).
4. Set up **Chromatic** for visual regression.

Before Phase 7 (security):

1. Add **Semgrep** + **Socket.dev** as GitHub Actions.
2. Add **Stagehand** for daily smoke runs.

Before Phase 8 (ship):

1. Sign up for **Tella** (or **Screen Studio** if you have a Mac).
2. Sign up for **Mintlify** + **Arcade**.
3. Toggle on **Vercel Speed Insights** in the Vercel dashboard.
- —

The Perfect Prompts (copy-paste into Antigravity or Trae)



How to use these: Paste each prompt into the IDE per the *IDE + model strategy* table above. For Phase 4 and Phase 7, use Antigravity's multi-agent panel and switch the model picker to **Opus 4.6**. For everything else, Trae with Sonnet 4.6 or GPT-5.3-Codex is plenty. Always **read the diff** before accepting — Antigravity and Trae both have visual review panels. After each phase, commit to git with a clear message. If something breaks, paste the full error output back in and ask for a fix.

IDE + model strategy by phase (April 2026)

Phase	IDE	Primary model	Why
3 (jobs CRUD)	Trae	GPT-5.3-Codex or Sonnet 4.6	Routine CRUD. Fast iteration over depth — don't waste Opus credits here.
4 (escrow state machine) ★	Antigravity	Claude Opus 4.6	80.8% SWE-bench, 1M context, multi-agent workflow. Race conditions and ledger atomicity demand the strongest reasoning available in April 2026.
5 (materials, disputes, admin)	Trae or Antigravity	Sonnet 4.6	Mid-complexity feature work. Sonnet 4.6 is the sweet spot — 79.6% SWE-bench at \$3/\$15 vs Opus \$5/\$25.
6 (PWA, polish, i18n)	Trae	GPT-5.3-Codex	Mechanical config + UI tweaks. Fast beats smart here.
7 (tests, security, RLS audit) ★	Antigravity	Claude Opus 4.6	Adversarial review of RLS policies, CSP, idempotency, race conditions. Pattern-matching models miss CVE-class issues here.
8 (deploy, Play Store, README polish)	Trae	Sonnet 4.6 or GPT-5.3-Codex	Mostly copy + config. Either model is fine.
Background (refactors, dep bumps, doc gen)	Claude Code (terminal)	Opus 4.6	Run autonomously while you sleep. The April 2026 superpower most devs aren't using yet.

Prompt 1 — Project bootstrap

You are a senior full-stack engineer. Bootstrap a production-grade Next.js 15 project called "pakka".

Requirements:

- Next.js 15 with App Router and TypeScript (strict mode)
- Tailwind CSS v4 + shadcn/ui (initialize with neutral base color)
- ESLint + Prettier configured
- Path alias @/* pointing to src/*
- Folder structure:
 - src/app (routes)
 - src/components/ui (shadcn)
 - src/components/features (feature components)
 - src/lib (utils, supabase client, types)
 - src/hooks
 - supabase/migrations
 - supabase/functions (edge functions)
- Install: @supabase/ssr, @supabase/supabase-js, zustand, @tanstack/react-query, zod, react-hook-form, @hookform/resolvers, sonner, lucide-react, date-fns, next-pwa
- Set up .env.local.example with all required vars
- Configure tailwind theme with brand colors:
 - primary: deep blue #0A2540
 - accent: emerald #10B981
 - Trust badge variants: bronze, silver, gold
- Create a basic layout with mobile-first viewport, max-width container 480px on mobile growing to full on desktop
- Add a global Toaster from sonner
- Configure next.config.ts for PWA (next-pwa) but leave service worker registration for later

Deliver: full file tree with all files. Use pnpm. Add a comprehensive README.md with setup instructions.

■ Prompt 2 — Database schema + RLS

Design and write the complete Supabase Postgres schema and RLS policies for Pakka.

Domain: two-sided marketplace for local construction/repair services with milestone-based escrow.

Tables needed:

1. **profiles** (id uuid FK auth.users, role enum 'client'|'worker'|'admin', full_name, phone, city, created_at)
2. **worker_profiles** (profile_id, kyc_status enum 'pending'|'verified'|'rejected', aadhaar_last4, pan_last4, selfie_url, categories text[], skill_tags text[], trust_tier enum 'bronze'|'silver'|'gold', rating numeric, jobs_completed int)
3. **wallets** (profile_id PK, locked_balance numeric, available_balance numeric, currency default 'INR')

4. `jobs` (id, client_id, worker_id nullable, title, description, category, location_text, lat, lng, total_budget, status enum 'draft'|'open'|'assigned'|'in_progress'|'completed'|'cancelled'|'disputed', created_at, accepted_at)
5. `milestones` (id, job_id, sequence int, title, description, amount, status enum 'pending'|'funded'|'submitted'|'approved'|'disputed'|'released'|'refunded', auto_release_at, submitted_at, approved_at)
6. `escrow\_ledger` (id, job_id, milestone_id, from_wallet, to_wallet, amount, type enum 'fund'|'release'|'refund'|'topup'|'withdraw', reference_id, created_at) – APPEND ONLY
7. `proofs` (id, milestone_id, type enum 'photo'|'video', storage_path, caption, geo_lat, geo_lng, taken_at, uploaded_at)
8. `disputes` (id, job_id, milestone_id, raised_by, reason, status enum 'open'|'mediating'|'resolved_client'|'resolved_worker'|'split', resolution_notes, resolved_by, created_at, resolved_at)
9. `materials` (id, job_id, vendor_name, item_name, qty, amount, status enum 'requested'|'paid'|'delivered', invoice_url)
10. `job\_applications` (id, job_id, worker_id, bid_amount, eta_days, message, status, created_at)
11. `notifications` (id, recipient_id, type, title, body, data jsonb, read_at, created_at)

Requirements:

- Use uuid PKs with `gen\_random\_uuid()`
- All timestamps timestampz default `now()`
- Add appropriate indexes (especially on FKs, status columns, location)
- Write RLS policies:
 - Users can read/update only their own profile
 - Clients see only their jobs; workers see open jobs in feed + jobs assigned to them
 - Wallet readable only by owner; never directly writable from `client` (only via SECURITY DEFINER functions)
 - `escrow\_ledger` is read-only to job participants; only Edge Functions can insert
 - admin role bypasses with policy USING `(is\_admin())`
- Create helper SQL functions:
 - `is\_admin()` returns boolean
 - `fund\_escrow(milestone\_id)` – atomic ledger entry + wallet update
 - `approve\_milestone(milestone\_id)` – atomic transfer locked→available on worker wallet, ledger entry, status update
 - `dispute\_milestone(milestone\_id, reason)`
 - `admin\_force\_release / admin\_refund`
- All money functions must be SECURITY DEFINER, transactional, and use SELECT FOR UPDATE on wallet rows to prevent race conditions

Deliver as numbered migration files in `supabase/migrations/` with clear naming. Include a `seed.sql` with 3 clients, 5 workers (mix of KYC states), 4 jobs in different states, sample milestones and ledger entries to demo every UI state.

Prompt 3 — Auth + OTP + role bifurcation

Implement passwordless phone OTP authentication using Supabase Auth.

Flow:

1. /login: phone input (+91 prefixed, 10 digits, validated with Zod and react-hook-form)
2. On submit, call `supabase.auth.signInWithOtp({ phone })`
3. /login/verify: 6-digit OTP input, auto-submit on completion (use `shadcn input-otp` component)
4. On success, check if profiles row exists for user
 - If not: redirect to /onboarding/role
 - If yes and role set: redirect to /client or /worker dashboard
5. /onboarding/role: two large cards (Client / Worker) with icons and one-line descriptions. Tap creates profiles row.
6. /onboarding/client: name + city → done
7. /onboarding/worker: multi-step KYC (name, Aadhaar last 4 + full upload, PAN, selfie via camera, categories multi-select, skill tags). Save `kyc_status='pending'`.

Technical:

- Use `@supabase/ssr` for proper server/client cookie handling in Next.js App Router
- Create `src/lib/supabase/client.ts`, `server.ts`, `middleware.ts`
- Add `middleware.ts` at root that protects all routes except `/login/*`, `/onboarding/*`, `/`
- Use Server Actions for OTP send/verify
- For KYC file uploads, upload to Supabase Storage bucket 'kyc' with private access; store `storage_path` only in DB
- Mobile-first: full-screen single-purpose pages with one large CTA at bottom
- Use brand colors. OTP screen has resend timer (60s).
- All forms use Zod schemas in `src/lib/schemas/`

For demo mode: if `env.DEMO_MODE=true`, accept OTP "123456" for any number without calling SMS provider. Make this clear with a small banner.

Prompt 3.5 — Jobs CRUD + browse + apply (Phase 3)

Implement Phase 3 of Pakka: job posting (client) + browse feed (worker) + applications + accept worker.

Context: Next.js 15 + TypeScript + Tailwind + shadcn/ui + Supabase project. Auth (phone OTP), role bifurcation, and KYC are already complete. The Postgres schema (jobs, milestones, materials, job_applications tables with full RLS) is already migrated. DO NOT modify migrations – only consume them.

Build these surfaces:

=== CLIENT: Post a job (/client/jobs/new) ===

Multi-step form using shadcn <Form>+ react-hook-form+ Zod. Steps:

1. Basics: title(3–80 chars), category(select from 10 seeded categories), description(20–2000 chars).
2. Location: free text city/area + optional lat/lng via browser geolocation with permission prompt; fallback to manual.
3. Budget: total_budget (₹500–₹500,000), currency INR fixed.
4. Milestones: dynamic list, min 1 max 8. Fields: title (3–60), description (optional), amount. MUST sum to total_budget – show running total + inline warning if over/under.
5. Materials (optional): dynamic list of vendor_name + item_name + qty + amount.
6. Review screen with everything, single "Post job" CTA.

On submit: Server Action that inserts jobs (status='open'), milestones (status='pen

ding'), materials (status='requested') in ONE transaction. Redirect to /client/jobs/[id].

=== CLIENT: Job detail (/client/jobs/[id]) ===

- Header: title, category badge, status badge, total budget, "posted X ago".
- Milestones list(read-only at this phase, status badges only).
- Materials list(read-only).
- Applications section: live count+ list of worker applications showing worker name, trust tier badge, bid amount, ETA days, message preview.
- Each application card has "Accept" button. Accept = Server Action that atomically: sets jobs.worker_id+ status='assigned'+ accepted_at=now(); sets selected application.status='accepted', other applications.status='rejected'. Then redirects to /client/jobs/[id]/fund.
- /client/jobs/[id]/fund is a placeholder page for this phase: "Escrow funding coming in the next phase" + back link.

=== CLIENT: Job list (/client/jobs) ===

- TanStack Query list of jobs WHERE client_id = current user, ORDER BY created_at DESC.
- Card per job: title, status badge, total budget, milestone count, application count, "posted X ago".
- Filter chips: all / open / assigned / completed.
- Empty state: illustration placeholder + "Post your first job" CTA.

=== WORKER: Browse feed (/worker/feed) ===

- TanStack Query infinite scroll list of jobs WHERE status='open'.
- Filter chips: category (multi-select; default to worker's own categories if KYC verified, else all 10), distance (5/10/25/50 km – only enabled if browser geolocation granted).
- Sort: newest / nearest / highest budget.
- Job card: title, category, location, total budget, "posted X ago", milestone count, "Apply" button (disabled with tooltip "Complete KYC to apply" if kyc_status != 'verified').
- Tap card → /worker/jobs/[id] detail.

=== WORKER: Job detail (/worker/jobs/[id]) ===

- Read-only view of job, milestones, materials. No application internals visible.
- "Apply" CTA at bottom(sticky on mobile). Disabled if worker already applied – show "Application sent" pill instead.
- Apply modal: bid_amount(must be within ±20% of total_budget – enforce in Zod), eta_days(1–60), message(0–500). Server Action inserts job_applications row.

=== WORKER: My applications(/worker/applications) ===

- List of applications by current worker, grouped by status(pending / accepted / rejected).
- Tap → job detail.

=== Realtime ===

- Subscribe to Supabase Realtime channel on job_applications. When a new row is inserted for a job where current user is the client, invalidate the applications query and toast: "New application from\[Worker Name]".

=== Technical requirements ===

- All mutations are Server Actions in src/app/_actions/. NEVER call Supabase directly from client components for writes.
- All queries use TanStack Query with sensible staleTime (30s for feeds, 10s for own data).
- Zod schemas in src/lib/schemas/jobs.ts shared between Server Action input and react-hook-form resolver.
- Every page has loading skeleton, empty state with illustration placeholder + CTA, and error boundary.
- Optimistic UI for Apply and Accept actions with rollback toast on error.
- Money values displayed as ₹X,XXX using Intl.NumberFormat('en-IN', { style: 'currency', currency: 'INR', maximumFractionDigits: 0 }).
- Mobile-first: full-screen forms with sticky CTA at bottom; max-width 640px centered on desktop.
- Reuse StatusBadge component if it exists, else create src/components/ui/status-badge.tsx covering: open, assigned, in_progress, completed, cancelled, disputed, pending, funded, submitted, approved, released, refunded.
- Add Sentry error capture inside every Server Action try/catch.
- Run pnpm typecheck and pnpm lint and fix all errors before declaring done.

=== Out of scope(DO NOT build in this phase) ===

- Escrow funding flow(Phase 4)
- Proof upload(Phase 5)
- Push notifications(Phase 6)
- Withdrawal(Phase 4)

At the end, output a list of files created/modified with a one-line description for each.

Prompt 4 — Escrow state machine (the critical one)

Implement the milestone escrow state machine – this is the financial core of the app and must be bulletproof.

State transitions (milestone.status):

- pending → funded: client funds milestone, atomic ledger entry, client.locked += amount, escrow_held += amount
- funded → submitted: worker uploads proof and clicks "Submit for review". Sets submitted_at and auto_release_at = now + 72h.
- submitted → approved: client clicks Approve. Atomically: escrow_held -= amount, worker.available += amount, ledger entry type=release. Trigger realtime notification.
- submitted → disputed: client clicks Dispute. Creates disputes row. Funds stay locked.
- submitted → approved: auto-release cron when auto_release_at < now() and no dispute exists.
- disputed → resolved_client: admin refunds. escrow_held -= amount, client.available += amount.
- disputed → resolved_worker: admin force-releases. Same as approve.
- disputed → split: admin splits X% to each (use SQL function with two ledger entries).

Implementation:

1. All transitions go through Postgres SECURITY DEFINER functions (already in schema). Never mutate wallets from client code.
2. Build /jobs/\[id]/milestones page showing each milestone as a card with status badge and contextual action button.

3. Status badges with brand colors:
 - pending: gray
 - funded: blue (Locked in escrow)
 - submitted: amber (Awaiting client review)
 - approved/released: emerald (Released to worker)
 - disputed: red
4. Use Supabase Realtime to subscribe to milestone changes – both client and worker UIs update instantly when status changes.
5. On every state change, insert a notifications row AND trigger a Web Push (use a Supabase Edge Function 'send-push' invoked by a Postgres trigger).
6. Approve button shows a confirmation modal: "Release ₹X,XXX to \[Worker Name]? This cannot be undone."
7. Wallet page shows two balances: "Available" and "Locked in escrow" with breakdown by job.
8. Add a Vitest test suite that simulates: fund → submit → approve, fund → submit → dispute → admin refund, fund → submit → auto-release after 72h. Verify ledger sums to zero across all wallets.
9. Add an Edge Function cron (every 5 min) that runs auto-release for eligible milestones.
10. Idempotency: every action accepts an Idempotency-Key header (UUID from client). Store in a transactions table. Reject duplicates.

UI: For every approve/dispute action, show optimistic UI but rollback on server error with a toast.

Prompt 5 — Proof uploads, materials, disputes, admin

Build the remaining feature surfaces.

1. Proof upload (worker submits milestone):
 - Camera capture OR file picker for photo + optional video
 - Capture geolocation at upload time, store with proof
 - Add caption input
 - Upload to Supabase Storage bucket 'proofs' (signed URLs only, never public)
 - Show uploaded proofs as a swipeable gallery on the milestone card
 - Refuse uploads > 25MB; compress photos client-side before upload
2. Materials direct purchase:
 - On a job, client can add materials. Mock vendor list seeded in DB.
 - Each material has status: requested → paid → delivered
 - Paying a material debits client.available, credits a 'platform_vendor_pool' wallet (mock)
 - Worker sees materials and can mark delivered with photo proof
3. Disputes:
 - Filing UI: reason dropdown + free text + optional evidence upload
 - Dispute thread page with chronological events (status changes, comments)
 - Both parties can add comments and evidence until admin closes
4. Admin panel (/admin, gated by profiles.role='admin'):
 - Dashboard with KPIs: total escrow held, open disputes, pending KYCs, GMV last 7 days
 - KYC review queue: approve/reject worker KYC with notes
 - Dispute queue with kill-switch buttons: Refund Client / Release to Worker / Split (with % slider)
 - User search and ban
 - Audit log table showing every admin action
5. Notifications:
 - In-app notification center (bell icon + drawer)
 - Web Push subscription on first login (with explicit prompt UX, not browser default)
 - Service worker handles background push and click-through to deep link

All lists use TanStack Query with proper invalidation. All mutations show optimistic UI + toasts. Every page has a loading skeleton and empty state.

Prompt 6 — PWA, polish, i18n

Make this a real installable PWA and production-polish the UI.

1. PWA configuration:
 - public/manifest.json with name, short_name, icons (192, 512, maskable), theme_color #0A2540, background_color, di

```
splay 'standalone', start_url '/'
```

- Generate icon set from a single 1024px source (provide a placeholder SVG of a shield + handshake)
- Service worker via next-pwa: cache-first for static, network-first for API, with offline fallback page
- Install prompt component (defer beforeinstallprompt event, show custom UI after 2nd visit)
- Splash screens for iOS

1. i18n:
 - Add next-intl. Languages: en, hi.
 - Translate all UI strings. Use translation keys in components.
 - Language switcher in profile menu, persist in profiles.preferred_language.
2. Polish pass:
 - Loading skeletons for every async surface
 - Empty states with illustration + CTA
 - Error boundaries with friendly messages and a 'Report' button (sends to Sentry)
 - All buttons min 48px tap target
 - Status badges consistent across the app – extract to a single <StatusBadge variant="..."/> component
 - Trust tier badges (Bronze/Silver/Gold) with distinct iconography
 - Confirm modals for all destructive/financial actions
 - Pull-to-refresh on feed pages
3. Accessibility:
 - All interactive elements keyboard-reachable
 - Proper ARIA labels on icon-only buttons
 - Color contrast AA minimum (verify with axe)
 - Focus rings preserved
4. Performance:
 - next/image for all images with proper sizes
 - Route-level code splitting
 - Run Lighthouse and fix anything below 90 in Performance, Accessibility, Best Practices, SEO, PWA

✓ Prompt 7 — Tests, observability, security audit

Make this production-ready with proper testing, observability, and security.

1. Testing:
 - Vitest unit tests for: every state machine SQL function (use a test Supabase instance), Zod schemas, wallet math
 - Playwright E2E covering: phone OTP signup (demo mode), client posts job, worker applies, client funds escrow, worker submits proof, client approves, worker withdraws – assert balances at every step
 - Run tests in GitHub Actions on every PR
2. Observability:
 - Sentry: configure for both client and server, source maps uploaded on deploy, capture user context (without PII)
 - PostHog: track events: signup_completed, kyc_submitted, job_posted, milestone_funded, milestone_approved, dispute_raised. Add session replay for non-PII screens only.
 - Structured logging on Edge Functions with request IDs
3. Security audit:
 - Verify every table has RLS enabled (write a script that fails CI if any table is missing RLS)
 - Confirm wallet/ledger writes are ONLY possible via SECURITY DEFINER functions
 - Add rate limiting to Edge Functions (10 req/min per user) using Supabase Postgres
 - Validate every input with Zod on both client and server
 - Sanitize file uploads: check MIME type server-side, scan for executable extensions
 - Add a Content Security Policy header
 - Enable Supabase email confirmations and password rules (even though we use OTP)
 - Privacy policy page generated from a template – required for Play Store
4. README upgrade:
 - Architecture diagram (Mermaid in markdown)
 - State machine diagram (Mermaid)
 - ER diagram (Mermaid)
 - Local setup steps
 - Demo credentials

- Embedded Loom video link
- Tech decisions section explaining why each choice was made

Prompt 8 — Deploy and ship to Play Store

Deploy to production and prepare Play Store submission.

1. Vercel deploy:

- Connect GitHub repo, **set** up production **branch** (main) and preview branches
- Add all env vars **in** Vercel dashboard **for prod** (separate Supabase prod project)
- Configure custom domain mypakka.vercel.app **with SSL**
- Add vercel.json **with** security **headers** (CSP, HSTS, X-Frame-Options, Referrer-Policy)

2. Supabase prod:

- New project, run all migrations, seed **with** minimal demo data
- Set up daily backups
- Configure **SMTP** **for** any transactional emails
- Set up Twilio/**MSG91** **for** real **SMS OTP** (or keep **DEMO_MODE=true** **if** budget is zero)

3. Play Store via **TWA**:

- Use PWABuilder.com → enter mypakka.vercel.app → download Android **package**
- Sign the **AAB** (PWABuilder generates a keystore – back it up immediately)
- Verify Digital Asset Links file is at **/.well-known/assetlinks.json** on the domain
- Test the **AAB** on a real device **with** bundletool

4. Play Console listing:

- App title: **"Pakka – Hire Skilled Workers Safely"**
- Short **description** (80 chars), full **description** (4000 chars) emphasizing escrow safety
- 8 phone **screenshots** (1080x1920) showing key flows
- Feature **graphic** (1024x500)
- App **icon** (512x512)
- Privacy policy **URL** (already deployed)
- Data safety form: be honest about phone, **KYC**, location, photos collected
- Content rating questionnaire
- Target audience: **18+**
- Choose internal testing track first → closed testing → production

5. Resume artifacts:

- **90-second Loom demo**: signup → post job → fund → submit → approve, narrated
- LinkedIn post **with** architecture diagram
- Blog post on dev.to or Hashnode covering: why **PWA+TWA**, how the escrow state machine works **with SQL**, how **RLS** replaces a traditional permissions layer
- Pin the GitHub repo on your profile **with** a great **README**

Generate the privacy policy page, the Play Store listing copy, and the LinkedIn post for me as separate files in the repo.

• -

Resume positioning (this matters more than you think)

Frame this as **open-source infrastructure**, not a “side project app.” The framing alone signals senior engineering thinking.

****Pakka — Open-source milestone escrow reference implementation** *(MIT, solo)***

- Architected and open-sourced a production-grade escrow ledger and state machine, intended as a reference for Indian marketplace builders. Live PWA demo (also on Play Store via TWA) showcases every flow with seeded data.

- Built on Next.js 15, TypeScript, Supabase (Postgres + RLS + Realtime + Edge Functions), deployed on Vercel + Play Store via TWA.

- Implemented a 7-state escrow state machine in SECURITY DEFINER Postgres functions with **SELECT FOR UPDATE** row locking, idempotent transactions, auto-release crons, and admin override. Ledger zero-sum invariant verified by Vitest property tests; full happy and dispute paths covered by Playwright.

- Phone-OTP auth with role bifurcation, KYC flow, photo/video proof with geo-tagging, realtime push notifications, Hindi/English i18n, offline-capable PWA.

- Documented architecture decisions (ADRs), tradeoffs, and the Razorpay Route production-swap path. 100% Lighthouse PWA score, RLS-enforced data isolation, Sentry + PostHog observability.

- Repo: github.com/yourname/pakka · Demo: mypakka.vercel.app · Writeup: dev.to/yourname/pakka-postgres-escrow

• -

Common pitfalls when vibe coding (avoid these)



Do not let the AI generate the financial state machine in TypeScript on the client. It **MUST** be Postgres SECURITY DEFINER functions with row locks. This is the #1 thing that distinguishes a toy from a real app.

- **Don't accept the first AI output blindly.** Always read the diff. Ask follow-up: “What edge cases did you miss?”
- **Don't skip RLS.** AI tools love to disable RLS “temporarily.” Never let that ship.
- **Don't store secrets in the repo.** Vercel and Supabase env vars only.
- **Don't fake the demo.** If it's mock money, label it. Recruiters reverse-engineer projects.
- **Don't skip tests for the state machine.** Even one Vitest file proving the ledger always balances is worth 10 features.
- **Commit after every prompt.** If the AI breaks something, **git reset --hard** is your friend.

• -

What to do right now (next 2 hours)

1. Sign up for Cursor, Vercel, Supabase, GitHub.
2. Create the empty repo.
3. Run **Prompt 1** in Cursor.
4. Commit. Push. See it deploy on Vercel.
5. Tomorrow: run Prompt 2.

You'll have a working app on a real domain by next weekend if you do one phase per day.

You've pasted the escrow state machine functions and seed SQL — let me know what you'd like me to do with them.

What should I do with the SQL you pasted?: Other: how would i know bro do what is best for the project

![image.png]

(Build%20Roadmap%20%E2%80%94%20Vibe%20Coding%20Pakka%20to%20Production/image.png)

![image.png]

(Build%20Roadmap%20%E2%80%94%20Vibe%20Coding%20Pakka%20to%20Production/image%201.png)

![image.png]

(Build%20Roadmap%20%E2%80%94%20Vibe%20Coding%20Pakka%20to%20Production/image%202.png)